University of Warwick

ES3B2 - Digital Systems Design

Design and Implementation of a Donkey Kong-Inspired Arcade Game on the Nexys 4 DDR FPGA Using Verilog HDL

u5572175 - Kirill Kolotilin

Module Leader: Dr. Eduardo Wachter

Submitted in part fulfillment of the requirements for the degree of BEng EE

Department of Electronic Engineering
University of Warwick, April 2026

Contents

List of Figures

1 Introduction

Field programmable gate arrays (FPGAs) are powerful technologies for designing digital systems that require real time operation, as they provide reconfigurable hardware capable of implementing numerous complex combinational and sequential circuits simultaneously (N. Sulaiman et al., 2009). Within such systems, video graphics array (VGA) is a widely adopted display standard that defines the analogue signaling protocol used to drive a monitor, specifying the precise timing of vertical and horizontal synchronisation pulses, pixel RGB data, and blanking intervals required to produce a stable raster image. As illustrated in Fig. 1, a VGA frame is formed by scanning pixels from left to right and top to bottom, with "H Sync" and "V Sync" pulses marking the start of each new line and frame respectively, while the front porch, back porch, and blanking regions provide the timing margins needed for correct display operation (VESA, 1994). The combination of FPGA reconfigurability and VGA output therefore provides a suitable platform for real-time graphics applications, since pixel generation, object rendering, and game logic can all be implemented directly in synchronous hardware rather than sequential software.

This report presents the design and implementation of a Donkey Kong-inspired arcade game developed entirely in Verilog HDL and deployed on the Digilent Nexys 4 DDR FPGA development board. The original Donkey Kong, created by the legendary designer Shigeru Miyamoto and released by Nintendo in 1981, is widely regarded as one of the most influential arcade games in history (Y. Wang and S. Pueblo, 2006). This project recreates its core gameplay elements in hardware, including fully animated sprites, rolling barrels, climbable ladders, collision effects, and score tracking, using custom hardware description rather than software. More specifically, game objects are rendered using combinational pixel-membership logic and bitmap ROMs, with game state managed through a hierarchy of finite state machines synchronised to the vertical blanking interval. The developed design outputs a 640×480 VGA signal at 60 Hz and utilizes several major Nexys 4 DDR peripherals, including push buttons for player input, slide switches for difficulty control, a seven-segment display for live score and elapsed time, and LEDs for player state indication.

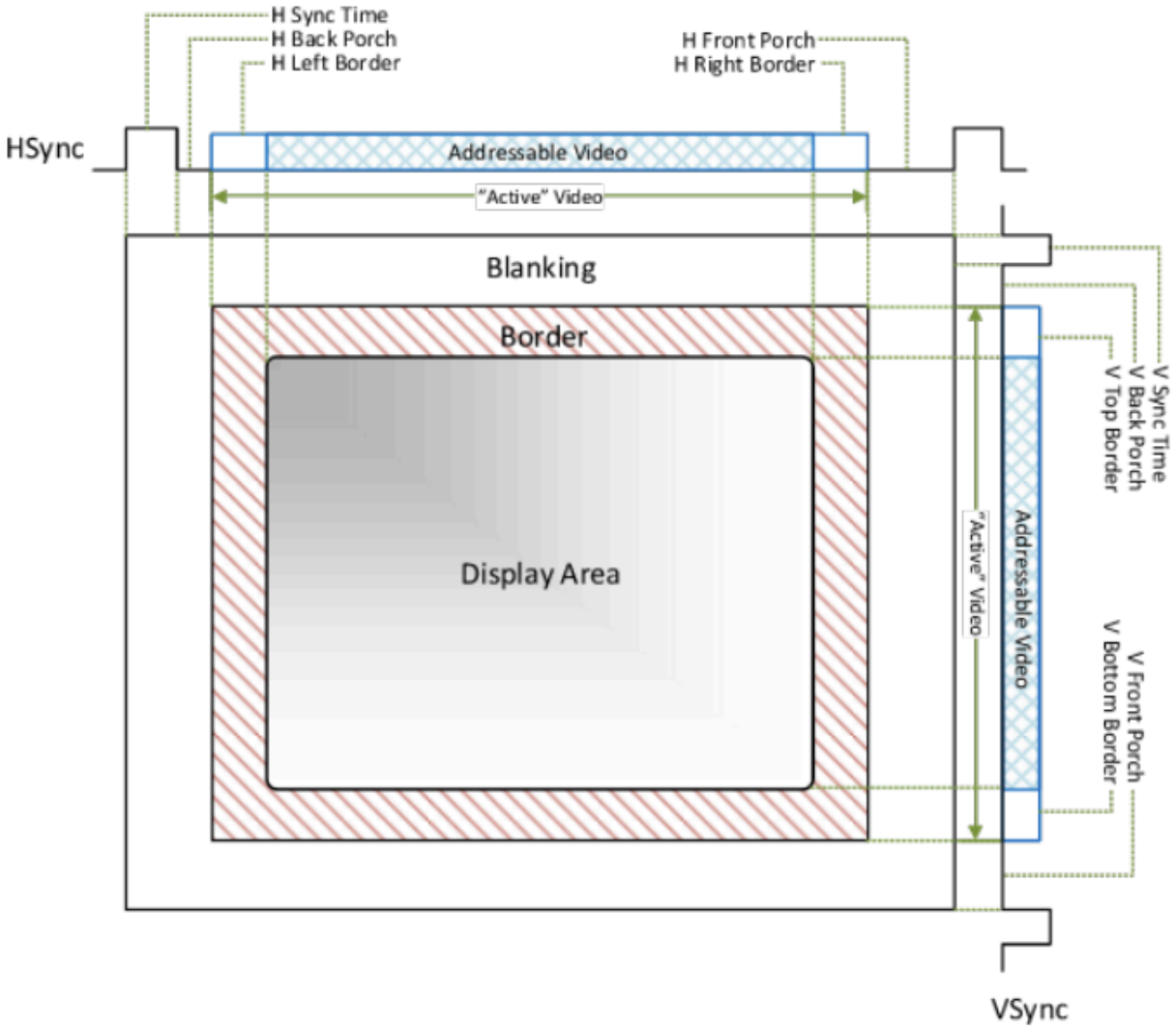

Figure 1: VGA frame timing showing active video, blanking intervals, and horizontal and vertical synchronisation pulses (E. Wachter, 2026).

2 Design

2.1 Game Overview

The developed game consists of two main characters: Mario and Donkey Kong. The objective of the game is for the player to navigate the Mario sprite from the bottom of the screen across seven sloped girder platforms, connected by eight ladders, to reach the trophy on the uppermost platform. A time-based score encourages fast completion. The obstacle to Mario's progression is the continuous stream of barrels spawned by Donkey Kong, which roll along the floors and must be avoided or jumped over. If Mario comes into contact with a barrel, he loses one life, and if he loses all the lives, the game is over.

2.2 Top-Level Architecture (dk_top)

The top-level module "dk_top" instantiates all sub-modules and manages signal routing between them. As illustrated in Fig. 2, the design is organised into three functional domains: timing generation, game logic and rendering, and peripheral output.

2.2.1 Timing Generation

The timing domain consists of the "clk_div" and "vga_sync" modules. The first module divides the 100 MHz board clock to produce a global 25 MHz pixel clock (clk_25), which satisfies the VGA timing requirement for a 640×480 display at 60 Hz. The second module then uses "clk_25" to generate the pixel coordinate signals "hpos" and "vpos", along with the outputs "hsync" and "vsync". Furthermore, the derived signal "frame_tick_top", computed combinationally as "vpos == 481 && hpos == 0", pulses for exactly one clock cycle at the start of each vertical blanking interval and is critical to the system, as it serves as the global enable signal for all game logic, ensuring that all state updates occur synchronously once per frame.

2.2.2 Game Logic

The game logic domain controls the main state of the system. The "player" module receives five push button inputs and produces position signals "player_x" and "player_y" consumed by the "barrels", "blood", "trophy", and "dust" modules. In addition, it generates status signals "pl_dying", "pl_is_jumping", and "pl_is_climbing" which are used by the event pulse detection logic and drive LED mapping outputs.

The event pulse detection logic performs edge detection on "pl_hit" and "pl_dying" to produce single-cycle pulses "hit_pulse" and "dying_pulse". This prevents the same collision from subtracting multiple lives. The "jump_pulse" is similarly edge-detected from the "br_jumped_over" signal to ensure each barrel jump scores exactly one hundred points.

The overall game state is governed by the "game_over", "you_win", and "lives" registers updated by several signals. When "hit_pulse" is asserted, the lives counter is decremented, and when reaching zero, "game_over" is set. Conversely, the "trophy" module asserts "player_wins" when reaching the goal, setting "you_win" and terminating further updates.

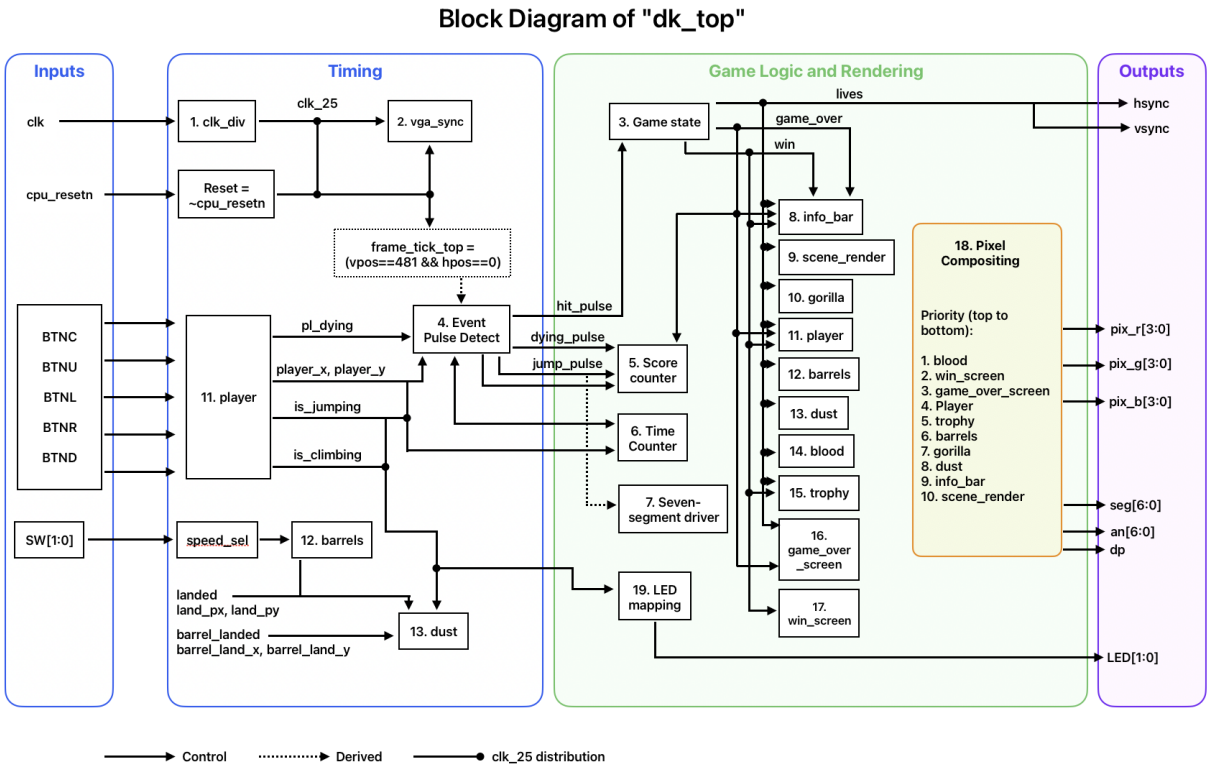

Figure 2: Top-level block diagram of the dk_top module.

The score counter increments by 100 on each successful jump over a barrel. In addition, it decreases periodically via "frame_tick_top", encouraging faster completion of the level while also preventing the score from going below zero. The time counter operates similarly, using a frame-based counter to increment "time_sec" once per second. These signals feed into the seven-segment driver, displaying eight digits at approximately 3 kHz.

The "barrels" module instantiates multiple independent moving obstacles, the speed of which is controlled via a skip counter mechanism by the "speed_sel" signal derived from "SW[1:0]". Both the "player" and "barrels" modules generate landing signals, which are routed to the "dust" module to trigger impact animations.

2.2.3 Rendering and Pixel Compositing

All rendering modules displayed in Fig. 2 receive "hpos", "vpos", and "active" and each produce a pixel colour. The pixel compositing block implements a priority multiplexer ensuring that foreground elements correctly overlay background graphics. This logic is purely combinational, making the scan-line rendering technique used in the design advantageous, as it enables real-time image generation without the need for frame buffering.

2.3 Scene Rendering (scene_render)

Pixel colours are generated by evaluating each pixel combinationally based on their co-ordinates, therefore, every clock cycle evaluates whether the current coordinate (hpos, vpos) falls within its boundaries, producing the colour within one 25 MHz clock period. The sloped girders are computed from "hpos" using an arithmetic right shift (slope = hpos >> 5) and the alternating slope layout of the platforms is achieved by adding or subtracting the slope from a set vertical position (y_rel). Furthermore, the zigzag filling of each girder is generated directly from the lower bits of "hpos", meaning that all the rendering is performed using only bitwise operations. Ladders are drawn by defining fixed horizontal and vertical pixel ranges, with rungs rendered by checking for periodicity in the lower bits of "vpos" (vpos[2:0] == 3'd0).

2.4 Information Bar (info_bar)

The information bar module occupies the top portion of the screen with a height of 100 pixels that span the full horizontal resolution. Separated from the game area by a solid orange border line at "vpos == 99", it renders the game title, the student identification numbers, a live score, and three heart icons representing the player's remaining lives. This region is defined by checking whether the current vertical position satisfies "vpos < 100" and when this condition is true, the background scene render is overridden and the info bar logic takes priority, ensuring that it is always visible regardless of game activity.

Text is rendered using a font ROM, implemented as a "reg" array containing 8×8 pixel bitmaps for each required character, including letters, digits, and the space character. Characters are displayed as 16×16 pixels by halving the pixel offset within the character cell before indexing the ROM (wire [2:0] xbit = rx[3:1]; wire [2:0] yrow = ry[3:1];). The latter is formed by concatenating the character index "cfid" with the row offset "yrow", and the correct character index for each screen position is selected by a "case" statement.

2.5 Gorilla Sprite (gorilla)

The "gorilla" module renders an animated gorilla sprite on the left side of the sixth floor, alongside a static pile of barrels. The gorilla cycles through "NORMAL", "REACH_LEFT",

"HOLD_BARREL", and "THROW" states, driven by a finite state machine that advances on "frame_tick". Their durations are parametrized (e.g. localparam [6:0] DUR_NORM = 7'd90;), producing an animation that conveys the gorilla picking up and throwing a barrel. The sprite is implemented as a 16×16 ROM and displayed as a 32×32 pixel sprite on screen. Three separate colours for the body, muzzle, and barrel mask are each stored as a 16-bit wide ROM array indexed by the current animation state and pixel row.

2.6 Player Sprite (player)

The "player" module utilises the same scanline rendering technique as the previous modules to maintain its position through registers "px" and "py", while player movement is governed by a four-state finite state machine (WALK, CLIMB, FALL, DYING), updated once per frame. In the "WALK" state the left/right buttons adjust "px" by ± 2 pixels while the vertical position snaps to the current floor's Y-coordinate. Pressing the centre button performs a jump by setting a negative vertical velocity "vy" and transitioning to "FALL" that is governed by a gravity accumulator "vy <= vy + 1" updating "py". Furthermore, when the player's centre pixel overlaps a ladder's x-range and the up or down button is pressed, the FSM transitions to CLIMB. Climbing moves "py" by ± 2 pixels until the player reaches the floor, at which point the FSM returns to "WALK".

2.7 Barrel Sprite (barrels & barrel_unit)

The "barrels" module manages a pool of 24 identical "barrel_unit" instances, each representing an independent barrel obstacle. In the design, each unit is either idle or rolling and an 8-bit counter generates a spawn pulse every 150 frames allocating the pulse to the lowest-indexed idle unit (if (spawn_cnt == 8'd149) spawn_cnt <= 0; else spawn_cnt <= spawn_cnt + 1;). The module's logic also guarantees that only one barrel spawns per cycle.

Each "barrel_unit" independently computes its own pixel output and collision flag, while "barrels" aggregates these outputs using bitwise OR reductions and asserts if any barrel collides with the player. Landing coordinates for score and particle effects are resolved through a conditional chain across the "bld" flags, selecting the position of whichever barrel landed that frame. The "speed_sel" input driven by "SW[0:1]" is passed uniformly to all barrel units without individual module logic, allowing to change the difficulty globally.

2.8 Dust Animation (dust)

The "dust" module provides a visual animation triggered when either the player or a barrel lands on a platform. The module instantiates two "dust_unit" sub-modules for each case, creating a five-phase sprite animation within a 32×16 pixel rendering area initiated by the "trigger" input, hence, starting 20-frame countdown timer. By right-shifting the timer by two bits ([2:0] phase = age[4:2]) each phase is held for four frames before advancing, rendering the progression from a dense central puff (phase0) through an expanding ring (phase1 - phase3) to scattered particles (phase4).

2.9 Blood Animation (blood)

The "blood" module produces a splatter animation when the player is hit by a barrel and it follows a similar sprite pattern as the "dust" module. The animation is divided into six phases over a 60-frame timer and it occurs in a 32×32 pixel area. The phase is determined by threshold comparisons on the timer value, dividing the 60 frames into six equal 10-frame intervals. As the bitmaps progress simulating blood dispersing outward, the colour transitions from bright red (R=0xF, G=0x1) to a darker red (R=0x8, G=0x0).

2.10 Trophy Sprite (trophy)

The "trophy" module renders a static win objective that is stored in ROM as an 8×8 bitmap and displayed as a 16×16 pixel trophy on the top platform. It is rendered in a gold colour (R=0xF, G=0xC, B=0x0). The win condition is evaluated by checking whether the player's bounding box overlaps the trophy's bounding box in "player_wins".

2.11 End Screens (game_over_screen & win_screen)

The "game_over_screen" and "win_screen" modules share an identical architecture, storing their character sets in a 64-entry font ROM and mapping characters through a combinational "case" statement: the first module maps indices 0–8, spelling "GAME OVER", while the second module maps indices 0–7 to "YOU WIN". A registered "always" block first checks the text pixel, then the box region, outputting yellow text; consequently, both modules produce a centred dark rectangle behind the text to ensure that the text is legible against the running game background.

3 Testing

Three testbenches were developed in the behavioural simulator of Vivado to verify the correct functional operation of the design. To accelerate simulation, testbenches modules held “vpos” and “hpos” at their “frame_tick_top” trigger values, 481 and 0 respectively. This decision allowed the finite state machine to update every clock cycle instead of once per full 800×525 VGA frame, reducing simulation time by a factor of 420,000.

3.1 VGA Sync: Specification Conformance Testing

The VGA timing testbench displayed in Fig. 3 verifies that the “vga_sync” module produces pulses at correct timing intervals against the 640×480 at 60 Hz standard. This testing compares the design’s outputs to the “VESA and Industry Standards” specification (VESA, 1994, p. 18) and it is the same methodology used in industry to validate IP blocks against interface standards for PCIe or DDR (H. Agoro and K. Reevs, 2025). The waveform achieved in the 5 s simulation confirms that “hpos” counts from 0 to 799 before wrapping, producing the expected 800 horizontal pixels (640 active, 16 front porch, 96 sync, 48 back porch). After “hpos” wraps, “vpos” increments by one and as it transitions from 489 to 490, the “vsync” signal goes low for exactly two lines (vpos 490 to 491). Similarly, “hsync” pulses low when “hpos” is between 656 and 751, validating the timing chain.

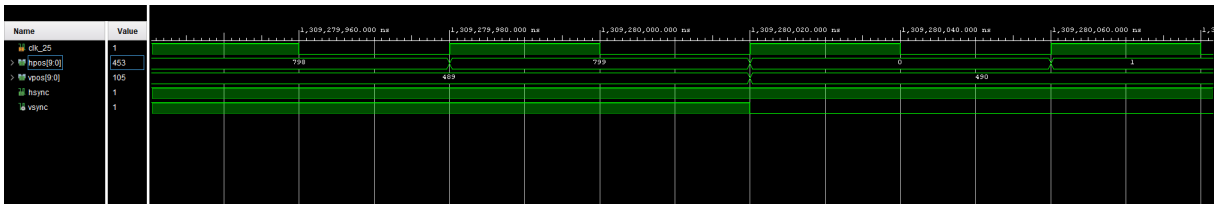

Figure 3: VGA sync testbench waveform.

3.2 Player: FSM State Analysis

The player testbench verifies the movement and jump behaviour of the player sprite by asserting controlled button inputs and observing the response of the position registers and FSM state. As shown in Fig. 4, when “btn_r” is asserted for a series of frames, “player_x” increases by 2 pixels each clock cycle (e.g. 478, 480, 482, ..., 498); at 1 ms, “btn_r” is toggled off and “player_x” holds a steady horizontal pixel value of 500, as expected.

Subsequently, the lower waveforms evaluate the player's state and changes in the FSM when "btn_c" is asserted to simulate a jump. The "state" waveform shows transitioning from 0 (WALK) to 2 (FALL), while "vy" increases by 1 each clock cycle (−6, −5, −4, ..., −1, 0, 1, ..., 7), confirming the correct operation of the gravity accumulator. Correspondingly, "player_y" first decreases from 434 to 413 (the player rises) and then returns to its initial value, producing the expected vertical jump. Furthermore, when "player_y" re-enters the floor's band, "state" and "vy" return to 0, confirming a successful landing.

This approach evaluates three FSM states (WALK, FALL, and landing state transition) and corresponds to FSM state coverage, a verification technique where the design is tested through important sequences of states, not just individual states or transitions (J. Barkley and S. Green, 2002).

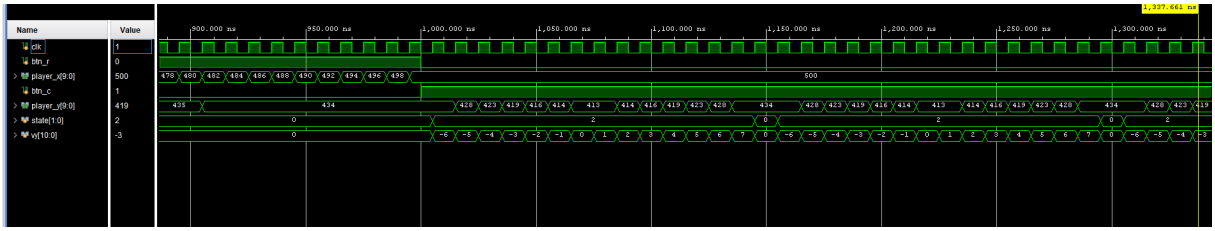

Figure 4: Player testbench waveform showing walk movement and jump mechanics.

3.3 Barrel Unit: Collision Verification

The barrel unit testbench, shown in Fig. 5, verifies the spawning mechanism and the collision detection between the barrel and the player. At 300 ns, the "spawn" signal is pulsed high causing the signal "is_idle" to turn low on the next rising clock edge (5 ns later), demonstrating synchronous operation. Once active, the barrel's internal position "bx" begins increasing from its spawn point (80), validating the ROLL state.

To test collisions, "player_x" and "player_y" are set to overlap the barrel's current position. As shown in the waveform, "hit_player" activates when the player's 8×16 bounding box overlaps the barrel's 8×8 bounding box (at 45 µs when "player_x" = 94). When the player's x and y pixel coordinates are in a non-overlapping position (100, 100), "hit_player" correctly deactivates, confirming the collision logic only fires during active overlap.

This testbench applies assertion-based verification, where expected design properties are specified and checked against actual behaviour. The manual waveform inspection verifies that "hit_player" asserts if "overlap" is active. Notably, accessing internal signals "bx" and "by" through the module hierarchy constitutes white-box testing, and it is observed to strengthen verification confidence.

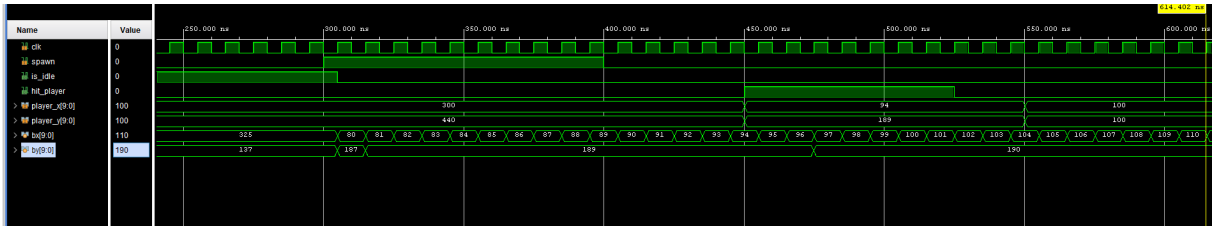

Figure 5: Barrel unit testbench waveform verifying collision detection.

4 Conclusion

This report presented the design and implementation of a Donkey Kong-inspired arcade game on the Nexys 4 DDR FPGA using Verilog HDL. The system integrates numerous modules synchronised to a single frame tick, implementing animated sprites, a 24-unit barrel pool, gravity-based jumping, and score tracking. Three testbenches verified the design using conformance testing, FSM state coverage, and assertion-based verification.

Key limitations include hardcoded sprite ROMs that make visual changes very tedious and predictable barrel spawning that experienced players can predict. Additionally, the game is limited to a single level with a fixed platform layout; storing multiple level configurations in block RAM and loading them on completion would strengthen the game's replayability. This project has improved the understanding of synchronous design and simulation-based verification, while deeper experience with memory-efficient design would be beneficial, since the current hardcoded ROM approach uses significant FPGA resources that could be reduced by using block RAM with external memory files.

5 References

[1] Sulaiman, N., Obaid, Z.A., Marhaban, M.H. and Hamidon, M.N., 2009. Design and Implementation of FPGA-Based Systems — A Review. *Australian Journal of Basic and Applied Sciences*. **3** (4). 3575–3596.

[2] VESA, 2013. *VESA and Industry Standards and Guidelines for Computer Display Monitor Timing (DMT)*. Retrieved April 26, 2026 from https://glenwing.github.io/docs/VESA-DMT-1.12.pdf.

[3] Wang, Y. and Pueblo, S., 2006. *6.111 Final Project*. Retrieved April 26, 2026 from https://fpga.mit.edu/videos/2006/team21/report.pdf.

[4] Wachter, E., 2026. *Lab Experiment 4 – VGA Output*. University of Warwick, School of Engineering. Retrieved April 26, 2026 from https://moodle.warwick.ac.uk/pluginfile.php/4121568/mod-label/intro/es3b2-lab04-22-23.pdf.

[5] Agoro, H. and Reevs, K., 2025. *Validation of DDR and PCIe Interoperability in Chiplet-Based Architectures*. ResearchGate. Retrieved April 26, 2026 from https://www.researchgate.net/publication/392311992_Validation_of_DDR_and-PCIe-Interoperability-in-Chiplet-Based_Architectures

[6] Barkley, J. and Green, S. 2002. *An Introduction to FSM Path Coverage*. TransEDA. Retrieved April 27, 2026 from https://uobdv.github.io/Design-Verification/Supplementary/An-Introduction_to_FSM-Path-Coverage.pdf

Appendix A: Source Code

A.1 — dk_top

```
`timescale 1ns / 1ps
module dk_top(
    input wire clk,
    input wire cpu_resetn,
    input wire BTNC, BTNU, BTNL, BTNR, BTND,
    input wire [1:0] SW,
    output wire [3:0] pix_r, pix_g, pix_b,
    output wire vsync, hsync,
    output wire [1:0] LED,
    output wire [6:0] seg,
    output wire [7:0] an,
    output wire dp
);
    // Clock and VGA timing
    wire clk_25, active;
    wire [9:0] hpos, vpos;
    wire reset = ~cpu_resetn;

    clk_div u_clk (.clk_100(clk), .clk_25(clk_25));
    vga_sync u_sync (.clk_25(clk_25), .hsync(hsync), .vsync(vsync),
        .active(active), .hpos(hpos), .vpos(vpos));

    // Frame tick: pulses once per VGA frame at start of vertical
       blanking
    wire frame_tick_top = (vpos == 10'd481 && hpos == 10'd0);
    // Barrel speed: 00=normal, 01=slow, 10=slower
    wire [1:0] speed_sel = SW[1] ? 2'd2 : SW[0] ? 2'd1 : 2'd0;

    // Game state registers
    reg [1:0] lives = 2'd3;
    reg game_over = 1'b0;
    reg you_win = 1'b0;

    wire pl_hit;
    wire pl_dying;
```

```
35      wire pl_is_jumping;
36      wire br_jumped_over;
37
38      // Edge detectors: produce single-cycle pulses to prevent repeated
            triggers
39      reg pl_hit_d = 1'b0;
40      always @(posedge clk_25) pl_hit_d <= pl_hit;
41      wire hit_pulse = pl_hit && !pl_hit_d;
42
43      reg pl_dying_d = 1'b0;
44      always @(posedge clk_25) pl_dying_d <= pl_dying;
45      wire dying_pulse = pl_dying && !pl_dying_d;
46
47      wire player_wins;
48
49      // Lives and win/lose logic
50      always @(posedge clk_25) begin
51          if (reset) begin
52              lives <= 2'd3;
53              game_over <= 1'b0;
54              you_win <= 1'b0;
55          end else begin
56              if (player_wins && !you_win && !game_over)
57                  you_win <= 1'b1;
58              if (hit_pulse && !game_over && !you_win) begin
59                  if (lives <= 2'd1) begin
60                      lives <= 2'd0;
61                      game_over <= 1'b1;
62                  end else
63                      lives <= lives - 2'd1;
64              end
65          end
66      end
67
68      // Score: +100 per barrel jump, decrements over time to reward speed
69      reg [13:0] score = 14'd0;
70      reg [2:0] sec_cnt = 3'd0;
71
72      reg br_jumped_d = 1'b0;
```

```
73      always @(posedge clk_25) br_jumped_d <= br_jumped_over;
74      wire jump_pulse = br_jumped_over && !br_jumped_d;
75
76      always @(posedge clk_25) begin
77          if (reset) begin
78              score <= 14'd0;
79              sec_cnt <= 3'd0;
80          end else begin
81              if (frame_tick_top) begin
82                  if (sec_cnt == 3'd5) sec_cnt <= 3'd0;
83                  else sec_cnt <= sec_cnt + 3'd1;
84              end
85              if (jump_pulse && !game_over && !you_win) begin
86                  if (score <= 14'd9899) score <= score + 14'd100;
87                  else score <= 14'd9999;
88              end else if (frame_tick_top && sec_cnt == 3'd5 &&
89                           score > 14'd0 && !game_over && !you_win) begin
90                  score <= score - 14'd1;
91              end
92          end
93      end
94
95      reg [13:0] time_sec = 14'd0;
96      reg [5:0] time_cnt = 6'd0;
97
98      always @(posedge clk_25) begin
99          if (reset) begin
100             time_sec <= 14'd0;
101             time_cnt <= 6'd0;
102         end else if (!game_over && !you_win) begin
103             if (frame_tick_top) begin
104                 if (time_cnt == 6'd59) begin
105                     time_cnt <= 6'd0;
106                     if (time_sec < 14'd9999) time_sec <= time_sec + 14'
                            d1;
107                 end else
108                     time_cnt <= time_cnt + 6'd1;
109             end
110         end
```

```
111     end
112
113     wire [3:0] t3 = time_sec / 14'd1000;
114     wire [3:0] t2 = (time_sec % 14'd1000) / 14'd100;
115     wire [3:0] t1 = (time_sec % 14'd100) / 14'd10;
116     wire [3:0] t0 = time_sec % 14'd10;
117     wire [3:0] s3 = score / 14'd1000;
118     wire [3:0] s2 = (score % 14'd1000) / 14'd100;
119     wire [3:0] s1 = (score % 14'd100) / 14'd10;
120     wire [3:0] s0 = score % 14'd10;
121
122     reg [16:0] mux_cnt = 17'd0;
123     always @(posedge clk_25) mux_cnt <= mux_cnt + 17'd1;
124     wire [2:0] digit_sel = mux_cnt[16:14];
125
126     reg [3:0] digit_val;
127     always @(*) case (digit_sel)
128         3'd7: digit_val = t3;
129         3'd6: digit_val = t2;
130         3'd5: digit_val = t1;
131         3'd4: digit_val = t0;
132         3'd3: digit_val = s3;
133         3'd2: digit_val = s2;
134         3'd1: digit_val = s1;
135         3'd0: digit_val = s0;
136         default: digit_val = 4'd0;
137     endcase
138
139     assign an = ~(8'b00000001 << digit_sel);
140
141     reg [6:0] seg_r;
142     always @(*) case (digit_val)
143         4'd0: seg_r = 7'b1000000;
144         4'd1: seg_r = 7'b1111001;
145         4'd2: seg_r = 7'b0100100;
146         4'd3: seg_r = 7'b0110000;
147         4'd4: seg_r = 7'b0011001;
148         4'd5: seg_r = 7'b0010010;
149         4'd6: seg_r = 7'b0000010;
```

```
150         4'd7: seg_r = 7'b1111000;
151         4'd8: seg_r = 7'b0000000;
152         4'd9: seg_r = 7'b0010000;
153         default: seg_r = 7'b1111111;
154     endcase
155     assign seg = seg_r;
156     assign dp = 1'b1;
157 
158     // Module instantiations
159     wire ib_in_bar;
160     wire [3:0] ib_r, ib_g, ib_b;
161     info_bar u_bar (
162         .clk(clk_25), .active(active),
163         .hpos(hpos), .vpos(vpos),
164         .lives(lives), .score(score),
165         .in_bar(ib_in_bar),
166         .r(ib_r), .g(ib_g), .b(ib_b));
167 
168     wire [3:0] sr_r, sr_g, sr_b;
169     scene_render u_render (
170         .clk(clk_25), .active(active),
171         .hpos(hpos), .vpos(vpos),
172         .r(sr_r), .g(sr_g), .b(sr_b));
173 
174     wire gor_active;
175     wire [3:0] gor_r, gor_g, gor_b;
176     gorilla u_gorilla (
177         .clk(clk_25), .active(active),
178         .hpos(hpos), .vpos(vpos),
179         .in_gorilla(gor_active),
180         .r(gor_r), .g(gor_g), .b(gor_b));
181 
182     wire pl_active;
183     wire [3:0] pl_r, pl_g, pl_b;
184     wire [9:0] pl_x, pl_y;
185     wire pl_landed;
186     wire [9:0] pl_lx, pl_ly;
187     wire pl_is_climbing;
188 
```

```
    player u_player (
        .clk(clk_25), .active(active),
        .hpos(hpos), .vpos(vpos),
        .btn_l(BTNL), .btn_r(BTNR),
        .btn_u(BTNU), .btn_d(BTND),
        .btn_c(BTNC),
        .hit(pl_hit), .full_rst(reset),
        .is_climbing(pl_is_climbing),
        .game_over(game_over | you_win),
        .player_x(pl_x), .player_y(pl_y),
        .dying(pl_dying),
        .is_jumping(pl_is_jumping),
        .in_player(pl_active),
        .landed(pl_landed), .land_px(pl_lx), .land_py(pl_ly),
        .r(pl_r), .g(pl_g), .b(pl_b));

    wire br_active;
    wire [3:0] br_r, br_g, br_b;
    wire br_landed;
    wire [9:0] br_lx, br_ly;

    barrels u_barrels (
        .clk(clk_25), .active(active),
        .hpos(hpos), .vpos(vpos),
        .player_x(pl_x), .player_y(pl_y),
        .is_jumping(pl_is_jumping),
        .in_barrel(br_active),
        .speed_sel(speed_sel),
        .hit_player(pl_hit),
        .barrel_landed(br_landed),
        .jumped_over(br_jumped_over),
        .barrel_land_x(br_lx), .barrel_land_y(br_ly),
        .r(br_r), .g(br_g), .b(br_b));

    wire dust_active;
    wire [3:0] dust_r, dust_g, dust_b;

    dust u_dust (
        .clk(clk_25), .active(active),
```

```
228         .hpos(hpos), .vpos(vpos),
229         .player_landed(pl_landed),
230         .player_lx(pl_lx), .player_ly(pl_ly),
231         .barrel_landed(br_landed),
232         .barrel_lx(br_lx), .barrel_ly(br_ly),
233         .in_dust(dust_active),
234         .r(dust_r), .g(dust_g), .b(dust_b));
235
236     wire bl_active;
237     wire [3:0] bl_r, bl_g, bl_b;
238
239     blood u_blood (
240         .clk(clk_25), .active(active),
241         .hpos(hpos), .vpos(vpos),
242         .trigger(dying_pulse),
243         .hit_x(pl_x + 10'd4), .hit_y(pl_y + 10'd8),
244         .in_blood(bl_active), .anim_done(),
245         .r(bl_r), .g(bl_g), .b(bl_b));
246
247     wire tr_active;
248     wire [3:0] tr_r, tr_g, tr_b;
249     trophy u_trophy (
250         .clk(clk_25), .active(active),
251         .hpos(hpos), .vpos(vpos),
252         .player_x(pl_x), .player_y(pl_y),
253         .in_trophy(tr_active),
254         .player_wins(player_wins),
255         .r(tr_r), .g(tr_g), .b(tr_b));
256
257     wire gos_active;
258     wire [3:0] gos_r, gos_g, gos_b;
259     game_over_screen u_gameover (
260         .clk(clk_25), .active(active),
261         .hpos(hpos), .vpos(vpos),
262         .game_over(game_over),
263         .in_screen(gos_active),
264         .r(gos_r), .g(gos_g), .b(gos_b));
265
266     wire ws_active;
```

```
267     wire [3:0] ws_r, ws_g, ws_b;
268     win_screen u_winscreen (
269         .clk(clk_25), .active(active),
270         .hpos(hpos), .vpos(vpos),
271         .you_win(you_win),
272         .in_screen(ws_active),
273         .r(ws_r), .g(ws_g), .b(ws_b));
274 
275     assign pix_r = bl_active ? bl_r :
276         ws_active ? ws_r : gos_active ? gos_r :
277         pl_active ? pl_r : tr_active ? tr_r :
278         br_active ? br_r : gor_active ? gor_r :
279         dust_active ? dust_r : ib_in_bar ? ib_r : sr_r;
280     assign pix_g = bl_active ? bl_g :
281         ws_active ? ws_g : gos_active ? gos_g :
282         pl_active ? pl_g : tr_active ? tr_g :
283         br_active ? br_g : gor_active ? gor_g :
284         dust_active ? dust_g : ib_in_bar ? ib_g : sr_g;
285     assign pix_b = bl_active ? bl_b :
286         ws_active ? ws_b : gos_active ? gos_b :
287         pl_active ? pl_b : tr_active ? tr_b :
288         br_active ? br_b : gor_active ? gor_b :
289         dust_active ? dust_b : ib_in_bar ? ib_b : sr_b;
290 
291     assign LED[0] = pl_is_jumping;
292     assign LED[1] = pl_is_climbing;
293 endmodule
```

A.2 — clk_div

```
`timescale 1ns / 1ps
module clk_div(
    input wire clk_100, // 100 MHz board clock
    output reg clk_25 // 25 MHz pixel clock
);
    reg [1:0] cnt = 0;
    always @(posedge clk_100) begin
        cnt <= cnt + 1;
        if (cnt == 1) begin
            clk_25 <= ~clk_25; // Toggle every 2 cycles
            cnt <= 0;
        end
    end
endmodule
```

A.3 — vga_sync

```
`timescale 1ns / 1ps
module vga_sync(
    input wire clk_25,
    output reg hsync, vsync,
    output reg active,
    output reg [9:0] hpos,
    output reg [9:0] vpos
);
    // 640x480 @ 60Hz
    parameter H_ACTIVE = 640, H_FP = 16, H_SYNC = 96, H_BP = 48;
    parameter V_ACTIVE = 480, V_FP = 10, V_SYNC = 2, V_BP = 33;

    reg [9:0] hcount = 0, vcount = 0;

    always @(posedge clk_25) begin
        // Horizontal counter
        if (hcount == H_ACTIVE + H_FP + H_SYNC + H_BP - 1) begin
            hcount <= 0;
            // Vertical counter
            if (vcount == V_ACTIVE + V_FP + V_SYNC + V_BP - 1)
                vcount <= 0;
            else
                vcount <= vcount + 1;
        end else
            hcount <= hcount + 1;

        hsync <= ~(hcount >= H_ACTIVE + H_FP && hcount < H_ACTIVE + H_FP
            + H_SYNC);
        vsync <= ~(vcount >= V_ACTIVE + V_FP && vcount < V_ACTIVE + V_FP
            + V_SYNC);

        active <= (hcount < H_ACTIVE && vcount < V_ACTIVE);
        hpos <= hcount;
        vpos <= vcount;
    end
endmodule
```

A.4 — info_bar

```
`timescale 1ns / 1ps
module info_bar(
    input wire clk,
    input wire active,
    input wire [9:0] hpos,
    input wire [9:0] vpos,
    input wire [1:0] lives,
    input wire [13:0] score,
    output reg in_bar,
    output reg [3:0] r,
    output reg [3:0] g,
    output reg [3:0] b
);
    localparam [11:0] BG = 12'h014;
    localparam [11:0] TEXT = 12'hFF0;
    localparam [11:0] BORDER = 12'hF80;
    localparam [11:0] HEART = 12'hF00;
    localparam [11:0] HEART_OFF = 12'h444;
    localparam [11:0] SCORE_COL = 12'h0FF;
    localparam [11:0] STITLE = 12'hFF0;

    // 21 characters stored as 8x8 bitmaps
    reg [7:0] font_rom [0:167];
    initial begin
        font_rom[0]=8'hFC; font_rom[1]=8'hC6; font_rom[2]=8'hC3;
           font_rom[3]=8'hC3;
        font_rom[4]=8'hC3; font_rom[5]=8'hC3; font_rom[6]=8'hC6;
           font_rom[7]=8'hFC;
        font_rom[8]=8'h3C; font_rom[9]=8'h66; font_rom[10]=8'hC3;
           font_rom[11]=8'hC3;
        font_rom[12]=8'hC3; font_rom[13]=8'hC3; font_rom[14]=8'h66;
           font_rom[15]=8'h3C;
        font_rom[16]=8'hC3; font_rom[17]=8'hE3; font_rom[18]=8'hF3;
           font_rom[19]=8'hDB;
        font_rom[20]=8'hCF; font_rom[21]=8'hC7; font_rom[22]=8'hC3;
           font_rom[23]=8'hC3;
```

```
31          font_rom[24]=8'hC6; font_rom[25]=8'hCC; font_rom[26]=8'hD8;
               font_rom[27]=8'hF0;
32          font_rom[28]=8'hF0; font_rom[29]=8'hD8; font_rom[30]=8'hCC;
               font_rom[31]=8'hC6;
33          font_rom[32]=8'hFE; font_rom[33]=8'hC0; font_rom[34]=8'hC0;
               font_rom[35]=8'hFC;
34          font_rom[36]=8'hC0; font_rom[37]=8'hC0; font_rom[38]=8'hC0;
               font_rom[39]=8'hFE;
35          font_rom[40]=8'hC3; font_rom[41]=8'hC3; font_rom[42]=8'h66;
               font_rom[43]=8'h3C;
36          font_rom[44]=8'h18; font_rom[45]=8'h18; font_rom[46]=8'h18;
               font_rom[47]=8'h18;
37          font_rom[48]=8'h00; font_rom[49]=8'h00; font_rom[50]=8'h00;
               font_rom[51]=8'h00;
38          font_rom[52]=8'h00; font_rom[53]=8'h00; font_rom[54]=8'h00;
               font_rom[55]=8'h00;
39          font_rom[56]=8'h3C; font_rom[57]=8'h66; font_rom[58]=8'hC0;
               font_rom[59]=8'hC0;
40          font_rom[60]=8'hCE; font_rom[61]=8'hC6; font_rom[62]=8'h66;
               font_rom[63]=8'h3C;
41          font_rom[64]=8'h3C; font_rom[65]=8'h66; font_rom[66]=8'hC3;
               font_rom[67]=8'hC3;
42          font_rom[68]=8'hC3; font_rom[69]=8'hC3; font_rom[70]=8'h66;
               font_rom[71]=8'h3C;
43          font_rom[72]=8'h18; font_rom[73]=8'h38; font_rom[74]=8'h78;
               font_rom[75]=8'h18;
44          font_rom[76]=8'h18; font_rom[77]=8'h18; font_rom[78]=8'h18;
               font_rom[79]=8'h7E;
45          font_rom[80]=8'h3C; font_rom[81]=8'h66; font_rom[82]=8'h06;
               font_rom[83]=8'h0C;
46          font_rom[84]=8'h18; font_rom[85]=8'h30; font_rom[86]=8'h60;
               font_rom[87]=8'hFE;
47          font_rom[88]=8'h3C; font_rom[89]=8'h66; font_rom[90]=8'h06;
               font_rom[91]=8'h1C;
48          font_rom[92]=8'h06; font_rom[93]=8'h06; font_rom[94]=8'h66;
               font_rom[95]=8'h3C;
49          font_rom[96]=8'h06; font_rom[97]=8'h0E; font_rom[98]=8'h1E;
               font_rom[99]=8'h36;
```

```
50          font_rom[100]=8'h66; font_rom[101]=8'hFE; font_rom[102]=8'h06;
                font_rom[103]=8'h06;
51          font_rom[104]=8'hFE; font_rom[105]=8'hC0; font_rom[106]=8'hC0;
                font_rom[107]=8'hFC;
52          font_rom[108]=8'h06; font_rom[109]=8'h06; font_rom[110]=8'hC6;
                font_rom[111]=8'h7C;
53          font_rom[112]=8'h3C; font_rom[113]=8'h66; font_rom[114]=8'hC0;
                font_rom[115]=8'hFC;
54          font_rom[116]=8'hC6; font_rom[117]=8'hC6; font_rom[118]=8'h66;
                font_rom[119]=8'h3C;
55          font_rom[120]=8'hFE; font_rom[121]=8'h06; font_rom[122]=8'h0C;
                font_rom[123]=8'h18;
56          font_rom[124]=8'h30; font_rom[125]=8'h30; font_rom[126]=8'h30;
                font_rom[127]=8'h30;
57          font_rom[128]=8'h3C; font_rom[129]=8'h66; font_rom[130]=8'h66;
                font_rom[131]=8'h3C;
58          font_rom[132]=8'h66; font_rom[133]=8'h66; font_rom[134]=8'h66;
                font_rom[135]=8'h3C;
59          font_rom[136]=8'h3C; font_rom[137]=8'h66; font_rom[138]=8'h66;
                font_rom[139]=8'h3E;
60          font_rom[140]=8'h06; font_rom[141]=8'h06; font_rom[142]=8'h66;
                font_rom[143]=8'h3C;
61          font_rom[144]=8'h7C; font_rom[145]=8'hC6; font_rom[146]=8'hC0;
                font_rom[147]=8'h7C;
62          font_rom[148]=8'h06; font_rom[149]=8'h06; font_rom[150]=8'hC6;
                font_rom[151]=8'h7C;
63          font_rom[152]=8'h3C; font_rom[153]=8'h66; font_rom[154]=8'hC0;
                font_rom[155]=8'hC0;
64          font_rom[156]=8'hC0; font_rom[157]=8'hC0; font_rom[158]=8'h66;
                font_rom[159]=8'h3C;
65          font_rom[160]=8'hFC; font_rom[161]=8'hC6; font_rom[162]=8'hC6;
                font_rom[163]=8'hFC;
66          font_rom[164]=8'hD8; font_rom[165]=8'hCC; font_rom[166]=8'hC6;
                font_rom[167]=8'hC6;
67      end
68
69      // grab each score digit
70      wire [13:0] score_cap = (score > 14'd9999) ? 14'd9999 : score;
71      wire [3:0] sd3 = score_cap / 14'd1000;
```

```
72      wire [3:0] sd2 = (score_cap % 14'd1000) / 14'd100;
73      wire [3:0] sd1 = (score_cap % 14'd100) / 14'd10;
74      wire [3:0] sd0 = score_cap % 14'd10;
75
76      // "SCORE" text at top left
77      wire in_slabel = (hpos >= 10'd10 && hpos < 10'd90 &&
78          vpos >= 10'd10 && vpos < 10'd26);
79      wire [9:0] slrx = hpos - 10'd10;
80      wire [9:0] slry = vpos - 10'd10;
81      wire [2:0] slcidx = slrx[6:4];
82      wire [2:0] slxbit = slrx[3:1];
83      wire [2:0] slyrow = slry[3:1];
84
85      reg [4:0] sl_cfid;
86      always @(*) case (slcidx)
87          3'd0: sl_cfid = 5'd18;
88          3'd1: sl_cfid = 5'd19;
89          3'd2: sl_cfid = 5'd1;
90          3'd3: sl_cfid = 5'd20;
91          3'd4: sl_cfid = 5'd4;
92          default: sl_cfid = 5'd6;
93      endcase
94      wire [7:0] sl_addr = {sl_cfid, slyrow};
95      wire sl_fpx = font_rom[sl_addr][7 - slxbit];
96
97      // numeric score below the label
98      wire in_sdigits = (hpos >= 10'd10 && hpos < 10'd74 &&
99          vpos >= 10'd30 && vpos < 10'd46);
100     wire [9:0] sdrx = hpos - 10'd10;
101     wire [9:0] sdry = vpos - 10'd30;
102     wire [1:0] sdcidx = sdrx[5:4];
103     wire [2:0] sdxbit = sdrx[3:1];
104     wire [2:0] sdyrow = sdry[3:1];
105
106     reg [3:0] sd_dig;
107     always @(*) case (sdcidx)
108         2'd0: sd_dig = sd3;
109         2'd1: sd_dig = sd2;
110         2'd2: sd_dig = sd1;
```

```
111         2'd3: sd_dig = sd0;
112         default: sd_dig = 4'd0;
113     endcase
114
115     // digit 0 starts at index 8
116     wire [4:0] sd_cfid = 5'd8 + sd_dig;
117     wire [7:0] sd_addr = {sd_cfid, sdyrow};
118     wire sd_fpx = font_rom[sd_addr][7 - sdxbit];
119     // "DONKEY KONG" centred
120     wire in_row1 = (hpos >= 10'd232 && hpos < 10'd408 &&
121         vpos >= 10'd17 && vpos < 10'd33);
122     wire [9:0] rx1 = hpos - 10'd232;
123     wire [9:0] ry1 = vpos - 10'd17;
124     wire [3:0] cidx1 = rx1[7:4];
125     wire [2:0] xbit1 = rx1[3:1];
126     wire [2:0] yrow1 = ry1[3:1];
127
128     reg [4:0] cfid1;
129     always @(*) case (cidx1)
130         4'd0: cfid1 = 5'd0;
131         4'd1: cfid1 = 5'd1;
132         4'd2: cfid1 = 5'd2;
133         4'd3: cfid1 = 5'd3;
134         4'd4: cfid1 = 5'd4;
135         4'd5: cfid1 = 5'd5;
136         4'd6: cfid1 = 5'd6;
137         4'd7: cfid1 = 5'd3;
138         4'd8: cfid1 = 5'd1;
139         4'd9: cfid1 = 5'd2;
140         4'd10: cfid1 = 5'd7;
141         default: cfid1 = 5'd6;
142     endcase
143     wire [7:0] addr1 = {cfid1, yrow1};
144     wire fpx1 = font_rom[addr1][7 - xbit1];
145     // student IDs
146     wire in_row2 = (hpos >= 10'd192 && hpos < 10'd448 &&
147         vpos >= 10'd62 && vpos < 10'd78);
148     wire [9:0] rx2 = hpos - 10'd192;
149     wire [9:0] ry2 = vpos - 10'd62;
```

```
150     wire [3:0] cidx2 = rx2[7:4];
151     wire [2:0] xbit2 = rx2[3:1];
152     wire [2:0] yrow2 = ry2[3:1];
153
154     reg [4:0] cfid2;
155     always @(*) case (cidx2)
156         4'd0: cfid2 = 5'd13;
157         4'd1: cfid2 = 5'd13;
158         4'd2: cfid2 = 5'd15;
159         4'd3: cfid2 = 5'd10;
160         4'd4: cfid2 = 5'd9;
161         4'd5: cfid2 = 5'd15;
162         4'd6: cfid2 = 5'd13;
163         4'd7: cfid2 = 5'd6;
164         4'd8: cfid2 = 5'd6;
165         4'd9: cfid2 = 5'd13;
166         4'd10: cfid2 = 5'd13;
167         4'd11: cfid2 = 5'd9;
168         4'd12: cfid2 = 5'd13;
169         4'd13: cfid2 = 5'd12;
170         4'd14: cfid2 = 5'd12;
171         4'd15: cfid2 = 5'd11;
172         default: cfid2 = 5'd6;
173     endcase
174     wire [7:0] addr2 = {cfid2, yrow2};
175     wire fpx2 = font_rom[addr2][7 - xbit2];
176
177     // heart icons
178     reg [7:0] heart_rom [0:7];
179     initial begin
180         heart_rom[0]=8'b00000000; heart_rom[1]=8'b01101100;
181         heart_rom[2]=8'b11111110; heart_rom[3]=8'b11111110;
182         heart_rom[4]=8'b01111100; heart_rom[5]=8'b00111000;
183         heart_rom[6]=8'b00010000; heart_rom[7]=8'b00000000;
184     end
185
186     localparam [9:0] HY = 10'd15;
187     localparam [9:0] H1X = 10'd568;
188     localparam [9:0] H2X = 10'd586;
```

```
    localparam [9:0] H3X = 10'd604;

    wire in_h1 = (hpos>=H1X && hpos<H1X+16 && vpos>=HY && vpos<HY+16);
    wire in_h2 = (hpos>=H2X && hpos<H2X+16 && vpos>=HY && vpos<HY+16);
    wire in_h3 = (hpos>=H3X && hpos<H3X+16 && vpos>=HY && vpos<HY+16);
    wire in_any_heart = in_h1 | in_h2 | in_h3;
    wire [9:0] hrx = in_h1?(hpos-H1X):in_h2?(hpos-H2X):(hpos-H3X);
    wire [9:0] hry = vpos - HY;
    wire [2:0] hcol = hrx[3:1];
    wire [2:0] hrow = hry[3:1];
    wire heart_px = heart_rom[hrow][7 - hcol];
    // red if alive, grey if lost
    wire heart1_on = (lives >= 2'd1);
    wire heart2_on = (lives >= 2'd2);
    wire heart3_on = (lives == 2'd3);
    wire [11:0] heart_colour =
        in_h1 ? (heart_px ? (heart1_on ? HEART : HEART_OFF) : BG) :
        in_h2 ? (heart_px ? (heart2_on ? HEART : HEART_OFF) : BG) :
        (heart_px ? (heart3_on ? HEART : HEART_OFF) : BG);
    // picking final colour
    wire bar_active = (vpos <= 10'd99);
    wire [11:0] colour =
        !active ? 12'h000 :
        !bar_active ? 12'h000 :
        (vpos == 10'd99) ? BORDER :
        (in_slabel && sl_fpx) ? STITLE :
        (in_sdigits && sd_fpx) ? SCORE_COL :
        (in_row1 && fpx1) ? TEXT :
        (in_row2 && fpx2) ? TEXT :
        in_any_heart ? heart_colour :
        BG;

    always @(posedge clk) begin
        in_bar <= bar_active;
        r <= colour[11:8];
        g <= colour[7:4];
        b <= colour[3:0];
    end
endmodule
```

A.5 — scene_render

```
1  `timescale 1ns / 1ps
2  module scene_render(
3      input wire clk,
4      input wire active,
5      input wire [9:0] hpos,
6      input wire [9:0] vpos,
7      output reg [3:0] r,
8      output reg [3:0] g,
9      output reg [3:0] b
10 );
11     localparam [11:0] BLACK = 12'h000;
12     localparam [11:0] RED = 12'hF00;
13     localparam [11:0] CYAN = 12'h0FF;
14
15     // slope for slanted platforms
16     wire [9:0] slope = hpos >> 5;
17
18     // even floors slope up, odd floors slope down
19     wire [9:0] f2_top = 10'd395 + slope;
20     wire [9:0] f4_top = 10'd295 + slope;
21     wire [9:0] f6_top = 10'd195 + slope;
22     wire [9:0] f1_top = (slope < 10'd465) ? (10'd465 - slope) : 10'd0;
23     wire [9:0] f3_top = (slope < 10'd360) ? (10'd360 - slope) : 10'd0;
24     wire [9:0] f5_top = (slope < 10'd260) ? (10'd260 - slope) : 10'd0;
25
26     // check if pixel vs platform
27     wire on_f1 = (hpos >= 10 && hpos <= 630 && vpos >= f1_top && vpos <=
           f1_top + 10);
28     wire on_f2 = (hpos >= 10 && hpos <= 580 && vpos >= f2_top && vpos <=
           f2_top + 10);
29     wire on_f3 = (hpos >= 40 && hpos <= 630 && vpos >= f3_top && vpos <=
           f3_top + 10);
30     wire on_f4 = (hpos >= 10 && hpos <= 580 && vpos >= f4_top && vpos <=
           f4_top + 10);
31     wire on_f5 = (hpos >= 40 && hpos <= 630 && vpos >= f5_top && vpos <=
           f5_top + 10);
```

```
32      wire on_f6 = (hpos >= 10 && hpos <= 580 && vpos >= f6_top && vpos <=
            f6_top + 10);
33      wire on_top = (hpos >= 250 && hpos <= 400 && vpos >= 145 && vpos <=
            155);
34      wire in_platform = on_f1|on_f2|on_f3|on_f4|on_f5|on_f6|on_top;
35
36      // how far down from the top
37      wire [9:0] yrel_f1 = vpos - f1_top;
38      wire [9:0] yrel_f2 = vpos - f2_top;
39      wire [9:0] yrel_f3 = vpos - f3_top;
40      wire [9:0] yrel_f4 = vpos - f4_top;
41      wire [9:0] yrel_f5 = vpos - f5_top;
42      wire [9:0] yrel_f6 = vpos - f6_top;
43      wire [9:0] yrel_top = vpos - 10'd145;
44
45      wire [3:0] y_rel =
46          on_f1 ? yrel_f1[3:0] :
47          on_f2 ? yrel_f2[3:0] :
48          on_f3 ? yrel_f3[3:0] :
49          on_f4 ? yrel_f4[3:0] :
50          on_f5 ? yrel_f5[3:0] :
51          on_f6 ? yrel_f6[3:0] :
52          on_top ? yrel_top[3:0] :
53          4'd0;
54
55      // solid lines at top and bottom of each girder
56      wire is_rail = in_platform && (y_rel == 4'd0 || y_rel == 4'd9);
57
58      // zigzag fill between the rails
59      wire in_web_zone = in_platform && (y_rel >= 4'd1) && (y_rel <= 4'd8)
            ;
60      wire [2:0] inner_y = y_rel[2:0] - 3'd1;
61      wire is_diag = in_web_zone && (
62          hpos[3] == 1'b0 ? (inner_y == (3'd7 - hpos[2:0]))
63                          : (inner_y == hpos[2:0]));
64      wire is_girder = is_rail || is_diag;
65
66      // ladders
67      wire in_ladder =
```

```
68          (hpos >= 500 && hpos <= 511 && vpos >= 420 && vpos <= 450 &&
69              (hpos <= 501 || hpos >= 510 || vpos[2:0] == 3'd0)) ||
70          (hpos >= 150 && hpos <= 161 && vpos >= 365 && vpos <= 398 &&
71              (hpos <= 151 || hpos >= 160 || vpos[2:0] == 3'd0)) ||
72          (hpos >= 400 && hpos <= 411 && vpos >= 316 && vpos <= 348 &&
73              (hpos <= 401 || hpos >= 410 || vpos[2:0] == 3'd0)) ||
74          (hpos >= 100 && hpos <= 111 && vpos >= 308 && vpos <= 356 &&
75              (hpos <= 101 || hpos >= 110 || vpos[2:0] == 3'd0)) ||
76          (hpos >= 530 && hpos <= 541 && vpos >= 254 && vpos <= 310 &&
77              (hpos <= 531 || hpos >= 540 || vpos[2:0] == 3'd0)) ||
78          (hpos >= 200 && hpos <= 211 && vpos >= 265 && vpos <= 300 &&
79              (hpos <= 201 || hpos >= 210 || vpos[2:0] == 3'd0)) ||
80          (hpos >= 450 && hpos <= 461 && vpos >= 219 && vpos <= 245 &&
81              (hpos <= 451 || hpos >= 460 || vpos[2:0] == 3'd0)) ||
82          (hpos >= 300 && hpos <= 311 && vpos >= 155 && vpos <= 203 &&
83              (hpos <= 301 || hpos >= 310 || vpos[2:0] == 3'd0));
84
85      // coloring
86      wire [11:0] colour =
87          !active ? BLACK :
88          is_girder ? RED :
89          in_ladder ? CYAN :
90          BLACK;
91
92      always @(posedge clk) begin
93          r <= colour[11:8];
94          g <= colour[7:4];
95          b <= colour[3:0];
96      end
97  endmodule
```

A.6 — gorilla

```
`timescale 1ns / 1ps
module gorilla(
    input wire clk,
    input wire active,
    input wire [9:0] hpos,
    input wire [9:0] vpos,
    output reg in_gorilla,
    output reg [3:0] r,
    output reg [3:0] g,
    output reg [3:0] b
);
    // position and size
    localparam [9:0] GX = 10'd44;
    localparam [9:0] GY = 10'd163;
    localparam [9:0] GW = 10'd32;
    localparam [9:0] GH = 10'd32;

    // animation states
    localparam NORMAL = 2'd0;
    localparam REACH_LEFT = 2'd1;
    localparam HOLD_BARREL = 2'd2;
    localparam THROW = 2'd3;

    // frame length
    localparam [6:0] DUR_NORM = 7'd90;
    localparam [6:0] DUR_REACH = 7'd20;
    localparam [6:0] DUR_HOLD = 7'd20;
    localparam [6:0] DUR_THROW = 7'd20;

    reg [1:0] anim_state = NORMAL;
    reg [6:0] anim_cnt = 7'd0;
    wire frame_tick = (vpos == 10'd481 && hpos == 10'd0);

    // cycle
    always @(posedge clk) begin
        if (frame_tick) begin
            case (anim_state)
```

```
38                NORMAL: if (anim_cnt>=DUR_NORM-1) begin anim_cnt<=0;
                      anim_state<=REACH_LEFT; end else anim_cnt<=anim_cnt
                      +1;
39                REACH_LEFT: if (anim_cnt>=DUR_REACH-1) begin anim_cnt
                      <=0; anim_state<=HOLD_BARREL; end else anim_cnt<=
                      anim_cnt+1;
40                HOLD_BARREL: if (anim_cnt>=DUR_HOLD-1) begin anim_cnt
                      <=0; anim_state<=THROW; end else anim_cnt<=anim_cnt
                      +1;
41                THROW: if (anim_cnt>=DUR_THROW-1) begin anim_cnt<=0;
                      anim_state<=NORMAL; end else anim_cnt<=anim_cnt+1;
42                default: anim_state <= NORMAL;
43            endcase
44        end
45    end
46
47    // 16x16 body bitmaps
48    reg [15:0] normal_rom [0:15];
49    reg [15:0] reach_rom [0:15];
50    reg [15:0] hold_rom [0:15];
51    reg [15:0] throw_rom [0:15];
52
53    // layers for colors
54    reg [15:0] muzzle_n [0:15];
55    reg [15:0] muzzle_r [0:15];
56    reg [15:0] muzzle_h [0:15];
57    reg [15:0] muzzle_t [0:15];
58    reg [15:0] barrel_h [0:15];
59    reg [15:0] barrel_t [0:15];
60
61    integer i;
62    initial begin
63        for (i=0; i<16; i=i+1) begin
64            muzzle_n[i]=16'h0; muzzle_r[i]=16'h0;
65            muzzle_h[i]=16'h0; muzzle_t[i]=16'h0;
66            barrel_h[i]=16'h0; barrel_t[i]=16'h0;
67        end
68        // normal
69        normal_rom[0]=16'h0FF0; normal_rom[1]=16'h1FF8;
```

```
        normal_rom[2]=16'h3FFC; normal_rom[3]=16'h3FFC;
        normal_rom[4]=16'h33CC; normal_rom[5]=16'h3FFC;
        normal_rom[6]=16'h1FF8; normal_rom[7]=16'h0FF0;
        normal_rom[8]=16'hFFFF; normal_rom[9]=16'hFFFF;
        normal_rom[10]=16'hCFF3; normal_rom[11]=16'hCFF3;
        normal_rom[12]=16'hCFF3; normal_rom[13]=16'h3FFC;
        normal_rom[14]=16'h1C38; normal_rom[15]=16'h1C38;
        muzzle_n[6]=16'h1FF8; muzzle_n[7]=16'h0FF0;
        // reaching left
        reach_rom[0]=16'h3FC0; reach_rom[1]=16'h7FE0;
        reach_rom[2]=16'hFFF0; reach_rom[3]=16'hFFF0;
        reach_rom[4]=16'hCF30; reach_rom[5]=16'hFFF0;
        reach_rom[6]=16'h7FE0; reach_rom[7]=16'h3FC0;
        reach_rom[8]=16'hFFFF; reach_rom[9]=16'hFFFE;
        reach_rom[10]=16'hFFFC; reach_rom[11]=16'hFFF8;
        reach_rom[12]=16'hFFF0; reach_rom[13]=16'h7FF8;
        reach_rom[14]=16'h1C38; reach_rom[15]=16'h1C38;
        muzzle_r[6]=16'h7FE0; muzzle_r[7]=16'h3FC0;
        // holding barrel
        hold_rom[0]=16'h0FF0; hold_rom[1]=16'h1FF8;
        hold_rom[2]=16'h3FFC; hold_rom[3]=16'h3FFC;
        hold_rom[4]=16'h33CC; hold_rom[5]=16'h3FFC;
        hold_rom[6]=16'h1FF8; hold_rom[7]=16'h0FF0;
        hold_rom[8]=16'hFFFF; hold_rom[9]=16'hC003;
        hold_rom[10]=16'hC003; hold_rom[11]=16'hC003;
        hold_rom[12]=16'hC003; hold_rom[13]=16'h3FFC;
        hold_rom[14]=16'h1C38; hold_rom[15]=16'h1C38;
        muzzle_h[6]=16'h1FF8; muzzle_h[7]=16'h0FF0;
        barrel_h[8]=16'h07E0; barrel_h[9]=16'h1FF8;
        barrel_h[10]=16'h1FF8; barrel_h[11]=16'h1FF8;
        barrel_h[12]=16'h07E0;
        // throwing right
        throw_rom[0]=16'h0FF0; throw_rom[1]=16'h1FF8;
        throw_rom[2]=16'h3FFC; throw_rom[3]=16'h3FFC;
        throw_rom[4]=16'h33CC; throw_rom[5]=16'h3FFC;
        throw_rom[6]=16'h1FF8; throw_rom[7]=16'h0FF0;
        throw_rom[8]=16'hFFFF; throw_rom[9]=16'hFFFF;
        throw_rom[10]=16'hCFFF; throw_rom[11]=16'hC0FF;
        throw_rom[12]=16'hE007; throw_rom[13]=16'h3FFC;
```

```
109         throw_rom[14]=16'h1C38; throw_rom[15]=16'h1C38;
110         muzzle_t[6]=16'h1FF8; muzzle_t[7]=16'h0FF0;
111         barrel_t[9]=16'h0007; barrel_t[10]=16'h000F;
112         barrel_t[11]=16'h000F; barrel_t[12]=16'h0007;
113     end
114
115     // stack of barrels next to the gorilla
116     reg [7:0] bp_rom [0:5];
117     initial begin
118         bp_rom[0]=8'b00111100; bp_rom[1]=8'b01111110;
119         bp_rom[2]=8'b11111111; bp_rom[3]=8'b11111111;
120         bp_rom[4]=8'b01111110; bp_rom[5]=8'b00111100;
121     end
122
123     wire in_bpcA = (hpos >= 10'd10 && hpos < 10'd18);
124     wire in_bpcB = (hpos >= 10'd20 && hpos < 10'd28);
125     wire in_bpr1 = (vpos >= 10'd171 && vpos < 10'd177);
126     wire in_bpr2 = (vpos >= 10'd177 && vpos < 10'd183);
127     wire in_bpr3 = (vpos >= 10'd183 && vpos < 10'd189);
128     wire in_bpr4 = (vpos >= 10'd189 && vpos < 10'd195);
129     wire in_bpbox = active && (in_bpcA || in_bpcB) &&
130         (in_bpr1 || in_bpr2 || in_bpr3 || in_bpr4);
131
132     wire [2:0] bpx = in_bpcA ? (hpos - 10'd10) : (hpos - 10'd20);
133     wire [2:0] bpy =
134         in_bpr1 ? (vpos - 10'd171) :
135         in_bpr2 ? (vpos - 10'd177) :
136         in_bpr3 ? (vpos - 10'd183) :
137         (vpos - 10'd189);
138     wire bp_pixel = in_bpbox && bp_rom[bpy][7 - bpx];
139
140     wire in_gbox = active &&
141         (hpos >= GX) && (hpos < GX + GW) &&
142         (vpos >= GY) && (vpos < GY + GH);
143     wire [9:0] grx = hpos - GX;
144     wire [9:0] gry = vpos - GY;
145     wire [3:0] gcol = grx[4:1]; // divide by 2 for scaling
146     wire [3:0] grow = gry[4:1];
147
```

```
148     reg [15:0] body_row, muzz_row, barr_row;
149     always @(*) begin
150         case (anim_state)
151             NORMAL: begin body_row=normal_rom[grow]; muzz_row=muzzle_n[
                    grow]; barr_row=16'h0; end
152             REACH_LEFT: begin body_row=reach_rom[grow]; muzz_row=
                    muzzle_r[grow]; barr_row=16'h0; end
153             HOLD_BARREL: begin body_row=hold_rom[grow]; muzz_row=
                    muzzle_h[grow]; barr_row=barrel_h[grow]; end
154             THROW: begin body_row=throw_rom[grow]; muzz_row=muzzle_t[
                    grow]; barr_row=barrel_t[grow]; end
155             default: begin body_row=16'h0; muzz_row=16'h0; barr_row=16'
                    h0; end
156         endcase
157     end
158
159     wire g_body = body_row[15 - gcol];
160     wire g_muzzle = muzz_row[15 - gcol];
161     wire g_barrel = barr_row[15 - gcol];
162
163     always @(posedge clk) begin
164         if (in_gbox && (g_body || g_barrel)) begin
165             in_gorilla <= 1'b1;
166             if (g_barrel) begin
167                 r <= 4'hF; g <= 4'h1; b <= 4'h0;
168             end else if (g_muzzle) begin
169                 r <= 4'hD; g <= 4'hA; b <= 4'h6;
170             end else begin
171                 r <= 4'h6; g <= 4'h3; b <= 4'h1;
172             end
173         end else if (bp_pixel) begin
174             in_gorilla <= 1'b1;
175             r <= 4'hF; g <= 4'h1; b <= 4'h0;
176         end else begin
177             in_gorilla <= 1'b0;
178             r <= 4'h0; g <= 4'h0; b <= 4'h0;
179         end
180     end
181 endmodule
```

A.7 — player

```
1  `timescale 1ns / 1ps
2  module player(
3      input wire clk,
4      input wire active,
5      input wire [9:0] hpos,
6      input wire [9:0] vpos,
7      input wire btn_l,
8      input wire btn_r,
9      input wire btn_u,
10     input wire btn_d,
11     input wire btn_c,
12     input wire hit,
13     input wire full_rst,
14     input wire game_over,
15     output wire [9:0] player_x,
16     output wire [9:0] player_y,
17     output wire in_player,
18     output wire dying,
19     output wire is_jumping,
20     output wire is_climbing,
21     output wire landed,
22     output wire [9:0] land_px,
23     output wire [9:0] land_py,
24     output wire [3:0] r,
25     output wire [3:0] g,
26     output wire [3:0] b
27 );
28     localparam [9:0] PW = 10'd8;
29     localparam [9:0] PH = 10'd16;
30
31     localparam WALK = 2'd0;
32     localparam CLIMB = 2'd1;
33     localparam FALL = 2'd2;
34     localparam DYING = 2'd3;
35
36     reg [1:0] state = WALK;
37     reg [9:0] px = 10'd300;
```

```
38      reg [9:0] py = 10'd440;
39      reg signed [10:0] vy = 11'sd0; // vertical velocity
40      reg [2:0] floor_id = 3'd1;
41      reg [3:0] lad_id = 4'd0;
42      reg [2:0] jump_floor = 3'd7; // prevents landing above launch floor
43      reg landed_r = 1'b0;
44      reg [5:0] dying_timer = 6'd0;
45      reg facing = 1'b0;
46      reg [1:0] walk_frame = 2'd0;
47      reg walk_fdir = 1'b0;
48      reg [2:0] walk_cnt = 3'd0;
49      reg climb_frame = 1'b0;
50      reg [2:0] climb_cnt = 3'd0;
51
52      wire frame_tick = (vpos == 10'd481 && hpos == 10'd0);
53      // floor y positions
54      wire [9:0] sl = px >> 5;
55      wire [9:0] fy1 = (sl < 10'd465) ? (10'd465 - sl) : 10'd0;
56      wire [9:0] fy2 = 10'd395 + sl;
57      wire [9:0] fy3 = (sl < 10'd360) ? (10'd360 - sl) : 10'd0;
58      wire [9:0] fy4 = 10'd295 + sl;
59      wire [9:0] fy5 = (sl < 10'd260) ? (10'd260 - sl) : 10'd0;
60      wire [9:0] fy6 = 10'd195 + sl;
61      localparam [9:0] fy7 = 10'd145;
62
63      // which floor the player is currently standing on
64      wire [9:0] cur_fy =
65          (floor_id==3'd1)?fy1:(floor_id==3'd2)?fy2:
66          (floor_id==3'd3)?fy3:(floor_id==3'd4)?fy4:
67          (floor_id==3'd5)?fy5:(floor_id==3'd6)?fy6:
68          (floor_id==3'd7)?fy7:py+PH;
69
70      wire [9:0] px_r = px + PW;
71      wire [9:0] feet = py + PH;
72
73      // horizontal bounds for each floor
74      wire in_f1x=(px>=10'd10&&px_r<=10'd630);
75      wire in_f2x=(px>=10'd10&&px_r<=10'd580);
76      wire in_f3x=(px>=10'd40&&px_r<=10'd630);
```

```
77      wire in_f4x=(px>=10'd10&&px_r<=10'd580);
78      wire in_f5x=(px>=10'd40&&px_r<=10'd630);
79      wire in_f6x=(px>=10'd10&&px_r<=10'd580);
80      wire in_f7x=(px>=10'd250&&px_r<=10'd400);
81
82      // landing detection
83      wire land_f1=in_f1x&&feet>=fy1&&feet<=fy1+10'd16;
84      wire land_f2=in_f2x&&feet>=fy2&&feet<=fy2+10'd16;
85      wire land_f3=in_f3x&&feet>=fy3&&feet<=fy3+10'd16;
86      wire land_f4=in_f4x&&feet>=fy4&&feet<=fy4+10'd16;
87      wire land_f5=in_f5x&&feet>=fy5&&feet<=fy5+10'd16;
88      wire land_f6=in_f6x&&feet>=fy6&&feet<=fy6+10'd16;
89      wire land_f7=in_f7x&&feet>=fy7&&feet<=fy7+10'd16;
90
91      // player centre
92      wire [9:0] pc = px + 4;
93
94      wire on_lad0=(pc>=500&&pc<=511)&&(py<450)&&(feet>420);
95      wire on_lad1=(pc>=150&&pc<=161)&&(py<398)&&(feet>365);
96      wire on_lad2=(pc>=400&&pc<=411)&&(py<348)&&(feet>316);
97      wire on_lad3=(pc>=100&&pc<=111)&&(py<356)&&(feet>308);
98      wire on_lad4=(pc>=530&&pc<=541)&&(py<310)&&(feet>254);
99      wire on_lad5=(pc>=200&&pc<=211)&&(py<300)&&(feet>265);
100     wire on_lad6=(pc>=450&&pc<=461)&&(py<245)&&(feet>219);
101     wire on_lad7=(pc>=300&&pc<=311)&&(py<203)&&(feet>155);
102     wire on_any_ladder=on_lad0|on_lad1|on_lad2|on_lad3|
103         on_lad4|on_lad5|on_lad6|on_lad7;
104     // snap to ladder centre
105     wire [9:0] lkp_snap=
106         on_lad0?10'd502:on_lad1?10'd152:on_lad2?10'd402:on_lad3?10'd102:
107         on_lad4?10'd532:on_lad5?10'd202:on_lad6?10'd452:10'd302;
108     wire [3:0] lkp_id=
109         on_lad0?4'd0:on_lad1?4'd1:on_lad2?4'd2:on_lad3?4'd3:
110         on_lad4?4'd4:on_lad5?4'd5:on_lad6?4'd6:4'd7;
111     // descend a ladder
112     wire can_desc_lad0=(pc>=500&&pc<=511)&&(floor_id==3'd2);
113     wire can_desc_lad1=(pc>=150&&pc<=161)&&(floor_id==3'd3);
114     wire can_desc_lad2=(pc>=400&&pc<=411)&&(floor_id==3'd4);
115     wire can_desc_lad3=(pc>=100&&pc<=111)&&(floor_id==3'd4);
```

```
116     wire can_desc_lad4=(pc>=530&&pc<=541)&&(floor_id==3'd5);
117     wire can_desc_lad5=(pc>=200&&pc<=211)&&(floor_id==3'd5);
118     wire can_desc_lad6=(pc>=450&&pc<=461)&&(floor_id==3'd6);
119     wire can_desc_lad7=(pc>=300&&pc<=311)&&(floor_id==3'd7);
120     wire can_desc_any=can_desc_lad0|can_desc_lad1|can_desc_lad2|
            can_desc_lad3|
121          can_desc_lad4|can_desc_lad5|can_desc_lad6|can_desc_lad7;
122
123     wire [9:0] desc_snap=
124         can_desc_lad0?10'd502:can_desc_lad1?10'd152:can_desc_lad2?10'
                d402:
125         can_desc_lad3?10'd102:can_desc_lad4?10'd532:can_desc_lad5?10'
                d202:
126         can_desc_lad6?10'd452:10'd302;
127     wire [3:0] desc_id=
128         can_desc_lad0?4'd0:can_desc_lad1?4'd1:can_desc_lad2?4'd2:
129         can_desc_lad3?4'd3:can_desc_lad4?4'd4:can_desc_lad5?4'd5:
130         can_desc_lad6?4'd6:4'd7;
131
132     // ladder geometry lookup
133     reg [9:0] lad_x, lad_ytop, lad_ybot;
134     reg [2:0] lad_fup, lad_fdn;
135     always @(*) begin
136         case(lad_id)
137             4'd0: begin lad_x=502; lad_ytop=420; lad_ybot=450; lad_fup
                    =3'd2; lad_fdn=3'd1; end
138             4'd1: begin lad_x=152; lad_ytop=365; lad_ybot=398; lad_fup
                    =3'd3; lad_fdn=3'd2; end
139             4'd2: begin lad_x=402; lad_ytop=316; lad_ybot=348; lad_fup
                    =3'd4; lad_fdn=3'd3; end
140             4'd3: begin lad_x=102; lad_ytop=308; lad_ybot=356; lad_fup
                    =3'd4; lad_fdn=3'd3; end
141             4'd4: begin lad_x=532; lad_ytop=254; lad_ybot=310; lad_fup
                    =3'd5; lad_fdn=3'd4; end
142             4'd5: begin lad_x=202; lad_ytop=265; lad_ybot=300; lad_fup
                    =3'd5; lad_fdn=3'd4; end
143             4'd6: begin lad_x=452; lad_ytop=219; lad_ybot=245; lad_fup
                    =3'd6; lad_fdn=3'd5; end
```

```
144             4'd7: begin lad_x=302; lad_ytop=155; lad_ybot=203; lad_fup
                    =3'd7; lad_fdn=3'd6; end
145             default: begin lad_x=0; lad_ytop=0; lad_ybot=480; lad_fup=3'
                    d1; lad_fdn=3'd1; end
146         endcase
147     end
148
149     // main FSM
150     always @(posedge clk) begin
151         if (frame_tick) begin
152             landed_r <= 1'b0;
153             if (full_rst) begin
154                 px<=10'd300; py<=10'd440; vy<=11'sd0;
155                 floor_id<=3'd1; jump_floor<=3'd7; state<=WALK;
156                 facing<=1'b0; walk_frame<=0; walk_cnt<=0;
157                 climb_frame<=0; climb_cnt<=0; dying_timer<=0;
158             end else if (hit && state != DYING) begin
159                 state <= DYING;
160                 dying_timer <= 6'd61;
161                 vy <= 11'sd0;
162             end else if (!game_over) begin
163                 case (state)
164                     WALK: begin
165                         // update facing direction
166                         if (btn_r) facing<=1'b0;
167                         if (btn_l) facing<=1'b1;
168                         // cycle walk animation frames
169                         if (btn_l || btn_r) begin
170                             if (walk_cnt >= 3'd6) begin
171                                 walk_cnt <= 0;
172                                 if (!walk_fdir) begin
173                                     if (walk_frame==2'd2) begin
                                            walk_fdir<=1; walk_frame<=2'd1;
                                            end
174                                     else walk_frame<=walk_frame+1;
175                                 end else begin
176                                     if (walk_frame==2'd0) begin
                                            walk_fdir<=0; walk_frame<=2'd1;
                                            end
```

```
                else walk_frame<=walk_frame-1;
            end
        end else walk_cnt<=walk_cnt+1;
    end else begin
        walk_frame<=2'd0; walk_cnt<=0;
    end
    // move left/right
    if (btn_l && px>10'd10) px<=px-10'd2;
    if (btn_r) begin
        case(floor_id)
            3'd1,3'd3,3'd5: if(px+PW+10'd2<=10'd630)
                px<=px+10'd2;
            default: px<=px+10'd2;
        endcase
    end
    // stick to floor
    py<=cur_fy-PH;
    // jump
    if (btn_c) begin
        vy<=-11'sd6; jump_floor<=floor_id; state<=
            FALL;
    end
    // grab ladder up
    if (on_any_ladder && btn_u) begin
        state<=CLIMB; px<=lkp_snap; lad_id<=lkp_id;
        climb_frame<=0; climb_cnt<=0;
    end
    // grab ladder down
    if (can_desc_any && btn_d) begin
        state<=CLIMB; px<=desc_snap; lad_id<=desc_id
            ;
        climb_frame<=0; climb_cnt<=0;
    end
    // off edge
    case(floor_id)
        3'd2: if(px_r>10'd580||px<10'd10) begin
            state<=FALL; vy<=0; jump_floor<=3'd7; end
        3'd3: if(px<10'd40) begin state<=FALL; vy
            <=0; jump_floor<=3'd7; end
```

```
211                     3'd4: if(px_r>10'd580||px<10'd10) begin
                            state<=FALL; vy<=0; jump_floor<=3'd7; end
212                     3'd5: if(px<10'd40) begin state<=FALL; vy
                            <=0; jump_floor<=3'd7; end
213                     3'd6: if(px_r>10'd580||px<10'd10) begin
                            state<=FALL; vy<=0; jump_floor<=3'd7; end
214                     3'd7: if(px_r>10'd400||px<10'd250) begin
                            state<=FALL; vy<=0; jump_floor<=3'd7; end
215                     default:;
216                 endcase
217             end
218             CLIMB: begin
219                 px<=lad_x; // lock to ladder centre
220                 if (btn_u||btn_d) begin
221                     if (climb_cnt>=3'd11) begin
222                         climb_cnt<=0; climb_frame<=~climb_frame;
223                     end else climb_cnt<=climb_cnt+1;
224                 end
225                 // step off
226                 if (btn_u) begin
227                     if (py<=lad_ytop) begin state<=WALK;
                            floor_id<=lad_fup; py<=lad_ytop-PH; end
228                     else py<=py-10'd2;
229                 end
230                 if (btn_d) begin
231                     if (py+PH>=lad_ybot) begin state<=WALK;
                            floor_id<=lad_fdn; py<=lad_ybot-PH; end
232                     else py<=py+10'd2;
233                 end
234             end
235             FALL: begin
236                 // gravity
237                 if (vy<11'sd12) vy<=vy+11'sd1;
238                 if (vy[10]) py<=py-(~vy[9:0]+10'd1); // going up
239                 else py<=py+vy[9:0]; // coming down
240                 if (btn_l&&px>10'd10) px<=px-10'd2;
241                 if (btn_r) px<=px+10'd2;
242                 // check
243                 if (!vy[10]) begin
```

```
                        if (land_f1&&jump_floor>=3'd1) begin py<=fy1
                            -PH; vy<=0; floor_id<=3'd1; state<=WALK;
                            landed_r<=1; end
                        else if (land_f2&&jump_floor>=3'd2) begin py
                            <=fy2-PH; vy<=0; floor_id<=3'd2; state<=
                            WALK; landed_r<=1; end
                        else if (land_f3&&jump_floor>=3'd3) begin py
                            <=fy3-PH; vy<=0; floor_id<=3'd3; state<=
                            WALK; landed_r<=1; end
                        else if (land_f4&&jump_floor>=3'd4) begin py
                            <=fy4-PH; vy<=0; floor_id<=3'd4; state<=
                            WALK; landed_r<=1; end
                        else if (land_f5&&jump_floor>=3'd5) begin py
                            <=fy5-PH; vy<=0; floor_id<=3'd5; state<=
                            WALK; landed_r<=1; end
                        else if (land_f6&&jump_floor>=3'd6) begin py
                            <=fy6-PH; vy<=0; floor_id<=3'd6; state<=
                            WALK; landed_r<=1; end
                        else if (land_f7&&jump_floor>=3'd7) begin py
                            <=fy7-PH; vy<=0; floor_id<=3'd7; state<=
                            WALK; landed_r<=1; end
                    end
                end
                DYING: begin
                    // count down then respawn at start
                    if (dying_timer > 0)
                        dying_timer <= dying_timer - 6'd1;
                    else begin
                        px<=10'd300; py<=10'd440; vy<=11'sd0;
                        floor_id<=3'd1; jump_floor<=3'd7; state<=
                            WALK;
                        facing<=1'b0; walk_frame<=0; walk_cnt<=0;
                        climb_frame<=0; climb_cnt<=0;
                    end
                end
            endcase
        end
    end
```

```
268
269     // colour layers
270     reg [7:0] rom_red [0:5][0:15];
271     reg [7:0] rom_blue [0:5][0:15];
272     reg [7:0] rom_skin [0:5][0:15];
273     reg [7:0] rom_brown [0:5][0:15];
274
275     integer fi, ri;
276     initial begin
277         for (fi=0; fi<6; fi=fi+1)
278             for (ri=0; ri<16; ri=ri+1) begin
279                 rom_red[fi][ri]=8'h00; rom_blue[fi][ri]=8'h00;
280                 rom_skin[fi][ri]=8'h00; rom_brown[fi][ri]=8'h00;
281             end
282         begin : walk_head
283             integer f;
284             for (f=0; f<3; f=f+1) begin
285                 rom_red[f][0]=8'b00111100;
286                 rom_red[f][1]=8'b01111110;
287                 rom_red[f][2]=8'b11110000;
288                 rom_skin[f][2]=8'b00001110;
289                 rom_skin[f][3]=8'b01111110;
290                 rom_red[f][3]=8'b10000000;
291                 rom_skin[f][4]=8'hDB;
292                 rom_skin[f][5]=8'b11000011;
293                 rom_brown[f][5]=8'b00111100;
294                 rom_brown[f][6]=8'b00011000;
295                 rom_skin[f][6]=8'b11100111;
296                 rom_red[f][7]=8'hFF;
297                 rom_skin[f][8]=8'b11000011;
298                 rom_blue[f][8]=8'b00111100;
299                 rom_blue[f][9]=8'b01111110;
300                 rom_blue[f][10]=8'b01111110;
301             end
302         end
303         // standing still
304         rom_blue[0][11]=8'b01100110;
305         rom_blue[0][12]=8'b01100110;
306         rom_blue[0][13]=8'b01100110;
```

```
307         rom_brown[0][14]=8'b01111110;
308         rom_brown[0][15]=8'b01111110;
309         // right leg forward
310         rom_blue[1][11]=8'b01110010;
311         rom_blue[1][12]=8'b01110000;
312         rom_blue[1][13]=8'b01100000;
313         rom_brown[1][13]=8'b11100000;
314         rom_brown[1][14]=8'b11110000;
315         rom_brown[1][15]=8'b11110000;
316         // left leg forward
317         rom_blue[2][11]=8'b01001110;
318         rom_blue[2][12]=8'b00001110;
319         rom_blue[2][13]=8'b00000110;
320         rom_brown[2][13]=8'b00001110;
321         rom_brown[2][14]=8'b00001111;
322         rom_brown[2][15]=8'b00001111;
323         // climb frame A: left arm up
324         rom_red[3][0]=8'b00111100;
325         rom_red[3][1]=8'b01111110;
326         rom_red[3][2]=8'b01111110;
327         rom_brown[3][3]=8'b01111110;
328         rom_brown[3][4]=8'b01111110;
329         rom_skin[3][5]=8'b11111111;
330         rom_red[3][6]=8'b10000001;
331         rom_skin[3][6]=8'b01000010;
332         rom_red[3][7]=8'hFF;
333         rom_skin[3][8]=8'b10000001;
334         rom_blue[3][8]=8'b01111110;
335         rom_blue[3][9]=8'b01111110;
336         rom_blue[3][10]=8'b01111110;
337         rom_blue[3][11]=8'b01100110;
338         rom_blue[3][12]=8'b01100010;
339         rom_blue[3][13]=8'b00000110;
340         rom_brown[3][13]=8'b00001110;
341         rom_brown[3][14]=8'b00001111;
342         rom_brown[3][15]=8'b00001111;
343         // climb frame B: right arm up
344         rom_red[4][0]=8'b00111100;
345         rom_red[4][1]=8'b01111110;
```

```
346         rom_red[4][2]=8'b01111110;
347         rom_brown[4][3]=8'b01111110;
348         rom_brown[4][4]=8'b01111110;
349         rom_skin[4][5]=8'b11111111;
350         rom_red[4][6]=8'b10000001;
351         rom_skin[4][6]=8'b01000010;
352         rom_red[4][7]=8'hFF;
353         rom_skin[4][8]=8'b10000001;
354         rom_blue[4][8]=8'b01111110;
355         rom_blue[4][9]=8'b01111110;
356         rom_blue[4][10]=8'b01111110;
357         rom_blue[4][11]=8'b01100110;
358         rom_blue[4][12]=8'b01000110;
359         rom_blue[4][13]=8'b01100000;
360         rom_brown[4][13]=8'b01110000;
361         rom_brown[4][14]=8'b11110000;
362         rom_brown[4][15]=8'b11110000;
363         // mid-air frame
364         rom_red[5][0]=8'b00111100;
365         rom_red[5][1]=8'b01111110;
366         rom_red[5][2]=8'b11110000;
367         rom_skin[5][2]=8'b00001110;
368         rom_skin[5][3]=8'b01111110;
369         rom_red[5][3]=8'b10000000;
370         rom_skin[5][4]=8'hDB;
371         rom_skin[5][5]=8'b11000011;
372         rom_brown[5][5]=8'b00111100;
373         rom_brown[5][6]=8'b00011000;
374         rom_skin[5][6]=8'b11100111;
375         rom_skin[5][7]=8'hFF;
376         rom_red[5][8]=8'b11000011;
377         rom_blue[5][8]=8'b00111100;
378         rom_blue[5][9]=8'b01111110;
379         rom_blue[5][10]=8'b01111110;
380         rom_blue[5][11]=8'b11000011;
381         rom_blue[5][12]=8'b11000011;
382         rom_blue[5][13]=8'b11000011;
383         rom_brown[5][14]=8'b11100111;
384         rom_brown[5][15]=8'b11110111;
```

```
385     end
386
387     // scan pixel
388     wire in_sbox = active &&
389         (hpos >= px) && (hpos < px + PW) &&
390         (vpos >= py) && (vpos < py + PH);
391     wire [2:0] col = hpos - px;
392     wire [3:0] row = vpos - py;
393     // flip horizontally when facing left
394     wire [2:0] rom_col = facing ? (3'd7 - col) : col;
395     // pick sprite frame based on current state
396     wire [2:0] frame_idx =
397         (state==DYING) ? 3'd0 :
398         (state==FALL) ? 3'd5 :
399         (state==CLIMB && climb_frame) ? 3'd4 :
400         (state==CLIMB) ? 3'd3 :
401         (walk_frame==2'd1) ? 3'd1 :
402         (walk_frame==2'd2) ? 3'd2 : 3'd0;
403     wire px_red = in_sbox && rom_red[frame_idx][row][rom_col];
404     wire px_blue = in_sbox && rom_blue[frame_idx][row][rom_col];
405     wire px_skin = in_sbox && rom_skin[frame_idx][row][rom_col];
406     wire px_brown = in_sbox && rom_brown[frame_idx][row][rom_col];
407
408     assign in_player = px_red || px_blue || px_skin || px_brown;
409     // colour priority
410     assign r = px_brown?4'h7:px_red?4'hF:px_blue?4'h0:px_skin?4'hF:4'h0;
411     assign g = px_brown?4'h3:px_red?4'h0:px_blue?4'h2:px_skin?4'hA:4'h0;
412     assign b = px_brown?4'h1:px_red?4'h0:px_blue?4'hF:px_skin?4'h6:4'h0;
413
414     assign player_x = px;
415     assign player_y = py;
416     assign dying = (state == DYING);
417     assign is_jumping = (state == FALL);
418     assign landed = landed_r;
419     assign land_px = px;
420     assign land_py = py + PH;
421     assign is_climbing = (state == CLIMB);
422 endmodule
```

A.8 — barrels

```
`timescale 1ns / 1ps
module barrels(
    input wire clk,
    input wire active,
    input wire [9:0] hpos,
    input wire [9:0] vpos,
    input wire [9:0] player_x,
    input wire [9:0] player_y,
    input wire is_jumping,
    input wire [1:0] speed_sel,
    output wire in_barrel,
    output wire hit_player,
    output wire barrel_landed,
    output wire jumped_over,
    output wire [9:0] barrel_land_x,
    output wire [9:0] barrel_land_y,
    output wire [3:0] r,
    output wire [3:0] g,
    output wire [3:0] b
);
    wire frame_tick = (vpos == 10'd481 && hpos == 10'd0);

    // new barrel every 150 frames
    reg [7:0] spawn_cnt = 0;
    wire spawn_pulse = frame_tick && (spawn_cnt == 8'd149);
    always @(posedge clk) begin
        if (frame_tick) begin
            if (spawn_cnt==8'd149) spawn_cnt<=0;
            else spawn_cnt<=spawn_cnt+1;
        end
    end

    // output buses
    wire [23:0] ib, hp, id, bld, jmp;
    wire [3:0] br [0:23];
    wire [3:0] bg [0:23];
    wire [3:0] bb [0:23];
```

```
38      wire [9:0] blx [0:23];
39      wire [9:0] bly [0:23];
40
41      // priority chain
42      wire spawn0 = spawn_pulse && id[0];
43      wire spawn1 = spawn_pulse && !id[0] && id[1];
44      wire spawn2 = spawn_pulse && !id[1:0] && id[2];
45      wire spawn3 = spawn_pulse && !id[2:0] && id[3];
46      wire spawn4 = spawn_pulse && !id[3:0] && id[4];
47      wire spawn5 = spawn_pulse && !id[4:0] && id[5];
48      wire spawn6 = spawn_pulse && !id[5:0] && id[6];
49      wire spawn7 = spawn_pulse && !id[6:0] && id[7];
50      wire spawn8 = spawn_pulse && !id[7:0] && id[8];
51      wire spawn9 = spawn_pulse && !id[8:0] && id[9];
52      wire spawn10 = spawn_pulse && !id[9:0] && id[10];
53      wire spawn11 = spawn_pulse && !id[10:0] && id[11];
54      wire spawn12 = spawn_pulse && !id[11:0] && id[12];
55      wire spawn13 = spawn_pulse && !id[12:0] && id[13];
56      wire spawn14 = spawn_pulse && !id[13:0] && id[14];
57      wire spawn15 = spawn_pulse && !id[14:0] && id[15];
58      wire spawn16 = spawn_pulse && !id[15:0] && id[16];
59      wire spawn17 = spawn_pulse && !id[16:0] && id[17];
60      wire spawn18 = spawn_pulse && !id[17:0] && id[18];
61      wire spawn19 = spawn_pulse && !id[18:0] && id[19];
62      wire spawn20 = spawn_pulse && !id[19:0] && id[20];
63      wire spawn21 = spawn_pulse && !id[20:0] && id[21];
64      wire spawn22 = spawn_pulse && !id[21:0] && id[22];
65      wire spawn23 = spawn_pulse && !id[22:0] && id[23];
66
67      // 24 independent barrel units
68      barrel_unit u0 (.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn0),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[0]),.hit_player(hp[0]),.is_idle(id[0]),.landed(bld
            [0]),.jumped_over(jmp[0]),.land_bx(blx[0]),.land_by(bly[0]),.r(br
            [0]),.g(bg[0]),.b(bb[0]));
69      barrel_unit u1 (.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn1),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
```

```
        in_barrel(ib[1]),.hit_player(hp[1]),.is_idle(id[1]),.landed(bld
        [1]),.jumped_over(jmp[1]),.land_bx(blx[1]),.land_by(bly[1]),.r(br
        [1]),.g(bg[1]),.b(bb[1]));
70      barrel_unit u2 (.clk(clk),.speed_sel(speed_sel),.active(active),.
        hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn2),.
        player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
        in_barrel(ib[2]),.hit_player(hp[2]),.is_idle(id[2]),.landed(bld
        [2]),.jumped_over(jmp[2]),.land_bx(blx[2]),.land_by(bly[2]),.r(br
        [2]),.g(bg[2]),.b(bb[2]));
71      barrel_unit u3 (.clk(clk),.speed_sel(speed_sel),.active(active),.
        hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn3),.
        player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
        in_barrel(ib[3]),.hit_player(hp[3]),.is_idle(id[3]),.landed(bld
        [3]),.jumped_over(jmp[3]),.land_bx(blx[3]),.land_by(bly[3]),.r(br
        [3]),.g(bg[3]),.b(bb[3]));
72      barrel_unit u4 (.clk(clk),.speed_sel(speed_sel),.active(active),.
        hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn4),.
        player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
        in_barrel(ib[4]),.hit_player(hp[4]),.is_idle(id[4]),.landed(bld
        [4]),.jumped_over(jmp[4]),.land_bx(blx[4]),.land_by(bly[4]),.r(br
        [4]),.g(bg[4]),.b(bb[4]));
73      barrel_unit u5 (.clk(clk),.speed_sel(speed_sel),.active(active),.
        hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn5),.
        player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
        in_barrel(ib[5]),.hit_player(hp[5]),.is_idle(id[5]),.landed(bld
        [5]),.jumped_over(jmp[5]),.land_bx(blx[5]),.land_by(bly[5]),.r(br
        [5]),.g(bg[5]),.b(bb[5]));
74      barrel_unit u6 (.clk(clk),.speed_sel(speed_sel),.active(active),.
        hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn6),.
        player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
        in_barrel(ib[6]),.hit_player(hp[6]),.is_idle(id[6]),.landed(bld
        [6]),.jumped_over(jmp[6]),.land_bx(blx[6]),.land_by(bly[6]),.r(br
        [6]),.g(bg[6]),.b(bb[6]));
75      barrel_unit u7 (.clk(clk),.speed_sel(speed_sel),.active(active),.
        hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn7),.
        player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
        in_barrel(ib[7]),.hit_player(hp[7]),.is_idle(id[7]),.landed(bld
        [7]),.jumped_over(jmp[7]),.land_bx(blx[7]),.land_by(bly[7]),.r(br
        [7]),.g(bg[7]),.b(bb[7]));
```

```
76      barrel_unit u8 (.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn8),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[8]),.hit_player(hp[8]),.is_idle(id[8]),.landed(bld
            [8]),.jumped_over(jmp[8]),.land_bx(blx[8]),.land_by(bly[8]),.r(br
            [8]),.g(bg[8]),.b(bb[8]));
77      barrel_unit u9 (.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn9),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[9]),.hit_player(hp[9]),.is_idle(id[9]),.landed(bld
            [9]),.jumped_over(jmp[9]),.land_bx(blx[9]),.land_by(bly[9]),.r(br
            [9]),.g(bg[9]),.b(bb[9]));
78      barrel_unit u10(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn10),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[10]),.hit_player(hp[10]),.is_idle(id[10]),.landed(
            bld[10]),.jumped_over(jmp[10]),.land_bx(blx[10]),.land_by(bly
            [10]),.r(br[10]),.g(bg[10]),.b(bb[10]));
79      barrel_unit u11(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn11),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[11]),.hit_player(hp[11]),.is_idle(id[11]),.landed(
            bld[11]),.jumped_over(jmp[11]),.land_bx(blx[11]),.land_by(bly
            [11]),.r(br[11]),.g(bg[11]),.b(bb[11]));
80      barrel_unit u12(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn12),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[12]),.hit_player(hp[12]),.is_idle(id[12]),.landed(
            bld[12]),.jumped_over(jmp[12]),.land_bx(blx[12]),.land_by(bly
            [12]),.r(br[12]),.g(bg[12]),.b(bb[12]));
81      barrel_unit u13(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn13),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[13]),.hit_player(hp[13]),.is_idle(id[13]),.landed(
            bld[13]),.jumped_over(jmp[13]),.land_bx(blx[13]),.land_by(bly
            [13]),.r(br[13]),.g(bg[13]),.b(bb[13]));
82      barrel_unit u14(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn14),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
```

```
        in_barrel(ib[14]),.hit_player(hp[14]),.is_idle(id[14]),.landed(
            bld[14]),.jumped_over(jmp[14]),.land_bx(blx[14]),.land_by(bly
            [14]),.r(br[14]),.g(bg[14]),.b(bb[14]));
83      barrel_unit u15(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn15),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[15]),.hit_player(hp[15]),.is_idle(id[15]),.landed(
            bld[15]),.jumped_over(jmp[15]),.land_bx(blx[15]),.land_by(bly
            [15]),.r(br[15]),.g(bg[15]),.b(bb[15]));
84      barrel_unit u16(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn16),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[16]),.hit_player(hp[16]),.is_idle(id[16]),.landed(
            bld[16]),.jumped_over(jmp[16]),.land_bx(blx[16]),.land_by(bly
            [16]),.r(br[16]),.g(bg[16]),.b(bb[16]));
85      barrel_unit u17(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn17),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[17]),.hit_player(hp[17]),.is_idle(id[17]),.landed(
            bld[17]),.jumped_over(jmp[17]),.land_bx(blx[17]),.land_by(bly
            [17]),.r(br[17]),.g(bg[17]),.b(bb[17]));
86      barrel_unit u18(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn18),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[18]),.hit_player(hp[18]),.is_idle(id[18]),.landed(
            bld[18]),.jumped_over(jmp[18]),.land_bx(blx[18]),.land_by(bly
            [18]),.r(br[18]),.g(bg[18]),.b(bb[18]));
87      barrel_unit u19(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn19),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[19]),.hit_player(hp[19]),.is_idle(id[19]),.landed(
            bld[19]),.jumped_over(jmp[19]),.land_bx(blx[19]),.land_by(bly
            [19]),.r(br[19]),.g(bg[19]),.b(bb[19]));
88      barrel_unit u20(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn20),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[20]),.hit_player(hp[20]),.is_idle(id[20]),.landed(
            bld[20]),.jumped_over(jmp[20]),.land_bx(blx[20]),.land_by(bly
            [20]),.r(br[20]),.g(bg[20]),.b(bb[20]));
```

```
89      barrel_unit u21(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn21),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[21]),.hit_player(hp[21]),.is_idle(id[21]),.landed(
            bld[21]),.jumped_over(jmp[21]),.land_bx(blx[21]),.land_by(bly
            [21]),.r(br[21]),.g(bg[21]),.b(bb[21]));
90      barrel_unit u22(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn22),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[22]),.hit_player(hp[22]),.is_idle(id[22]),.landed(
            bld[22]),.jumped_over(jmp[22]),.land_bx(blx[22]),.land_by(bly
            [22]),.r(br[22]),.g(bg[22]),.b(bb[22]));
91      barrel_unit u23(.clk(clk),.speed_sel(speed_sel),.active(active),.
            hpos(hpos),.vpos(vpos),.frame_tick(frame_tick),.spawn(spawn23),.
            player_x(player_x),.player_y(player_y),.is_jumping(is_jumping),.
            in_barrel(ib[23]),.hit_player(hp[23]),.is_idle(id[23]),.landed(
            bld[23]),.jumped_over(jmp[23]),.land_bx(blx[23]),.land_by(bly
            [23]),.r(br[23]),.g(bg[23]),.b(bb[23]));
92
93      assign in_barrel = |ib;
94      assign hit_player = |hp;
95      assign barrel_landed = |bld;
96      assign jumped_over = |jmp;
97
98      // dust position
99      assign barrel_land_x =
100         bld[0]?blx[0]:bld[1]?blx[1]:bld[2]?blx[2]:bld[3]?blx[3]:
101         bld[4]?blx[4]:bld[5]?blx[5]:bld[6]?blx[6]:bld[7]?blx[7]:
102         bld[8]?blx[8]:bld[9]?blx[9]:bld[10]?blx[10]:bld[11]?blx[11]:
103         bld[12]?blx[12]:bld[13]?blx[13]:bld[14]?blx[14]:bld[15]?blx[15]:
104         bld[16]?blx[16]:bld[17]?blx[17]:bld[18]?blx[18]:bld[19]?blx[19]:
105         bld[20]?blx[20]:bld[21]?blx[21]:bld[22]?blx[22]:bld[23]?blx
                [23]:10'd0;
106     assign barrel_land_y =
107         bld[0]?bly[0]:bld[1]?bly[1]:bld[2]?bly[2]:bld[3]?bly[3]:
108         bld[4]?bly[4]:bld[5]?bly[5]:bld[6]?bly[6]:bld[7]?bly[7]:
109         bld[8]?bly[8]:bld[9]?bly[9]:bld[10]?bly[10]:bld[11]?bly[11]:
110         bld[12]?bly[12]:bld[13]?bly[13]:bld[14]?bly[14]:bld[15]?bly[15]:
111         bld[16]?bly[16]:bld[17]?bly[17]:bld[18]?bly[18]:bld[19]?bly[19]:
```

```
112         bld[20]?bly[20]:bld[21]?bly[21]:bld[22]?bly[22]:bld[23]?bly
                [23]:10'd0;
113
114     // OR Combination
115     assign r = br[0]|br[1]|br[2]|br[3]|br[4]|br[5]|br[6]|br[7]|
116         br[8]|br[9]|br[10]|br[11]|br[12]|br[13]|br[14]|br[15]|
117         br[16]|br[17]|br[18]|br[19]|br[20]|br[21]|br[22]|br[23];
118     assign g = bg[0]|bg[1]|bg[2]|bg[3]|bg[4]|bg[5]|bg[6]|bg[7]|
119         bg[8]|bg[9]|bg[10]|bg[11]|bg[12]|bg[13]|bg[14]|bg[15]|
120         bg[16]|bg[17]|bg[18]|bg[19]|bg[20]|bg[21]|bg[22]|bg[23];
121     assign b = bb[0]|bb[1]|bb[2]|bb[3]|bb[4]|bb[5]|bb[6]|bb[7]|
122         bb[8]|bb[9]|bb[10]|bb[11]|bb[12]|bb[13]|bb[14]|bb[15]|
123         bb[16]|bb[17]|bb[18]|bb[19]|bb[20]|bb[21]|bb[22]|bb[23];
124 endmodule
```

A.9 — dust

```
`timescale 1ns / 1ps
module dust(
    input wire clk,
    input wire active,
    input wire [9:0] hpos,
    input wire [9:0] vpos,
    input wire player_landed,
    input wire [9:0] player_lx,
    input wire [9:0] player_ly,
    input wire barrel_landed,
    input wire [9:0] barrel_lx,
    input wire [9:0] barrel_ly,
    output wire in_dust,
    output wire [3:0] r,
    output wire [3:0] g,
    output wire [3:0] b
);
    wire frame_tick = (vpos == 10'd481 && hpos == 10'd0);

    wire pd_active;
    wire [3:0] pd_r, pd_g, pd_b;
    wire bd_active;
    wire [3:0] bd_r, bd_g, bd_b;

    // dust for player & barrel
    dust_unit u_player_dust(
        .clk(clk), .active(active), .hpos(hpos), .vpos(vpos),
        .frame_tick(frame_tick),
        .trigger(player_landed),
        .land_x(player_lx), .land_y(player_ly),
        .in_dust(pd_active), .r(pd_r), .g(pd_g), .b(pd_b));

    dust_unit u_barrel_dust(
        .clk(clk), .active(active), .hpos(hpos), .vpos(vpos),
        .frame_tick(frame_tick),
        .trigger(barrel_landed),
        .land_x(barrel_lx), .land_y(barrel_ly),
```

```
38          .in_dust(bd_active), .r(bd_r), .g(bd_g), .b(bd_b));
39
40      // player dust priority
41      assign in_dust = pd_active | bd_active;
42      assign r = pd_active ? pd_r : bd_active ? bd_r : 4'h0;
43      assign g = pd_active ? pd_g : bd_active ? bd_g : 4'h0;
44      assign b = pd_active ? pd_b : bd_active ? bd_b : 4'h0;
45  endmodule
```

A.10 — blood

```
1  `timescale 1ns / 1ps
2  module blood(
3      input wire clk,
4      input wire active,
5      input wire [9:0] hpos,
6      input wire [9:0] vpos,
7      input wire trigger,
8      input wire [9:0] hit_x,
9      input wire [9:0] hit_y,
10     output wire in_blood,
11     output wire anim_done,
12     output wire [3:0] r,
13     output wire [3:0] g,
14     output wire [3:0] b
15 );
16     wire frame_tick = (vpos == 10'd481 && hpos == 10'd0);
17
18     reg [9:0] bx = 10'd0;
19     reg [9:0] by = 10'd0;
20     reg [5:0] timer = 6'd0;
21     reg trigger_latch = 1'b0;
22
23     // latch trigger instantly
24     always @(posedge clk) begin
25         if (trigger)
26             trigger_latch <= 1'b1;
27         if (frame_tick) begin
28             if (trigger_latch || trigger) begin
29                 bx <= hit_x;
30                 by <= hit_y;
31                 timer <= 6'd60; // 1 second at 60fps
32                 trigger_latch <= 1'b0;
33             end else if (timer > 0)
34                 timer <= timer - 6'd1;
35         end
36     end
37
```

```
38      // 6 phases
39      wire [2:0] phase = (timer >= 6'd50) ? 3'd0 :
40          (timer >= 6'd40) ? 3'd1 :
41          (timer >= 6'd30) ? 3'd2 :
42          (timer >= 6'd20) ? 3'd3 :
43          (timer >= 6'd10) ? 3'd4 : 3'd5;
44
45      assign anim_done = (timer == 6'd0);
46
47      localparam [9:0] DW = 10'd32;
48      localparam [9:0] DH = 10'd32;
49
50      // centre the 32x32 area on the hit point
51      wire [9:0] rx = (bx >= 10'd16) ? bx - 10'd16 : 10'd0;
52      wire [9:0] ry = (by >= 10'd16) ? by - 10'd16 : 10'd0;
53
54      wire in_box = active && (timer > 0) &&
55          (hpos >= rx) && (hpos < rx + DW) &&
56          (vpos >= ry) && (vpos < ry + DH);
57
58      wire [4:0] dpx = hpos - rx;
59      wire [4:0] dpy = vpos - ry;
60
61      // 6 bitmaps
62      reg [31:0] ph0 [0:31];
63      reg [31:0] ph1 [0:31];
64      reg [31:0] ph2 [0:31];
65      reg [31:0] ph3 [0:31];
66      reg [31:0] ph4 [0:31];
67      reg [31:0] ph5 [0:31];
68
69      integer i;
70      initial begin
71          for (i=0; i<32; i=i+1) begin
72              ph0[i]=32'h0; ph1[i]=32'h0; ph2[i]=32'h0;
73              ph3[i]=32'h0; ph4[i]=32'h0; ph5[i]=32'h0;
74          end
75          // dense central burst
76          ph0[12]=32'b00000000000111111111100000000000;
```

```
77          ph0[13]=32'b00000000011111111111111000000000;
78          ph0[14]=32'b00000000111111111111111100000000;
79          ph0[15]=32'b00000001111111111111111110000000;
80          ph0[16]=32'b00000001111111111111111110000000;
81          ph0[17]=32'b00000000111111111111111100000000;
82          ph0[18]=32'b00000000011111111111111000000000;
83          ph0[19]=32'b00000000000111111111100000000000;
84          // ring expanding outward
85          ph1[8]=32'b00000000000001100000000000000000;
86          ph1[9]=32'b00000000000110011000000000000000;
87          ph1[10]=32'b00000000011000000110000000000000;
88          ph1[11]=32'b00000000110000000001100000000000;
89          ph1[12]=32'b00000001100000000000110000000000;
90          ph1[13]=32'b00000011000000000000011000000000;
91          ph1[14]=32'b00000010000000000000001000000000;
92          ph1[15]=32'b00000110000000000000000110000000;
93          ph1[16]=32'b00000110000000000000000110000000;
94          ph1[17]=32'b00000010000000000000001000000000;
95          ph1[18]=32'b00000011000000000000011000000000;
96          ph1[19]=32'b00000001100000000000110000000000;
97          ph1[20]=32'b00000000110000000001100000000000;
98          ph1[21]=32'b00000000011000000110000000000000;
99          ph1[22]=32'b00000000000110011000000000000000;
100         ph1[23]=32'b00000000000001100000000000000000;
101         ph1[7]=32'b00000000010000000000100000000000;
102         ph1[24]=32'b00000000010000000000100000000000;
103         // wider spread
104         ph2[5]=32'b00000000001000000000010000000000;
105         ph2[6]=32'b00000000010000000000001000000000;
106         ph2[7]=32'b00000001000000000000000100000000;
107         ph2[8]=32'b00000010000000000000000010000000;
108         ph2[9]=32'b00000100000000000000000001000000;
109         ph2[10]=32'b00001000000000000000000000100000;
110         ph2[11]=32'b00010000000000000000000000010000;
111         ph2[12]=32'b00100000000000000000000000001000;
112         ph2[13]=32'b01000000000000000000000000000100;
113         ph2[15]=32'b10000000000000000000000000000010;
114         ph2[16]=32'b10000000000000000000000000000010;
115         ph2[18]=32'b01000000000000000000000000000100;
```

```
116         ph2[19]=32'b00100000000000000000000000001000;
117         ph2[20]=32'b00010000000000000000000000010000;
118         ph2[21]=32'b00001000000000000000000000100000;
119         ph2[22]=32'b00000100000000000000000001000000;
120         ph2[23]=32'b00000010000000000000000010000000;
121         ph2[24]=32'b00000001000000000000000100000000;
122         ph2[25]=32'b00000000010000000000001000000000;
123         ph2[26]=32'b00000000001000000000010000000000;
124         ph2[3]=32'b00000000000001100000000000000000;
125         ph2[4]=32'b00000000000011000000000000000000;
126         ph2[27]=32'b00000000000001100000000000000000;
127         ph2[28]=32'b00000000000011000000000000000000;
128         // sparse droplets
129         ph3[2]=32'b00000000000011000000000000000000;
130         ph3[3]=32'b00000000001000000000010000000000;
131         ph3[4]=32'b00000001000000000000000100000000;
132         ph3[6]=32'b00000100000000000000000000010000;
133         ph3[9]=32'b01000000000000000000000000000010;
134         ph3[10]=32'b10000000000000000000000000000001;
135         ph3[14]=32'b10000000000000000000000000000001;
136         ph3[17]=32'b10000000000000000000000000000001;
137         ph3[21]=32'b10000000000000000000000000000001;
138         ph3[22]=32'b01000000000000000000000000000010;
139         ph3[25]=32'b00000100000000000000000000010000;
140         ph3[27]=32'b00000001000000000000000100000000;
141         ph3[28]=32'b00000000001000000000010000000000;
142         ph3[29]=32'b00000000000011000000000000000000;
143         // nearly gone
144         ph4[1]=32'b00000000001000000000000000000000;
145         ph4[4]=32'b00000100000000000000000000000000;
146         ph4[7]=32'b10000000000000000000000000000000;
147         ph4[10]=32'b00000000000000000000000000000001;
148         ph4[15]=32'b00000000000000000000000000000001;
149         ph4[16]=32'b10000000000000000000000000000000;
150         ph4[21]=32'b00000000000000000000000000000001;
151         ph4[24]=32'b10000000000000000000000000000000;
152         ph4[27]=32'b00000100000000000000000000000000;
153         ph4[30]=32'b00000000001000000000000000000000;
154         // last few dots
```

```
        ph5[1]=32'b00000000000010000000000000000000;
        ph5[30]=32'b00000000000010000000000000000000;
        ph5[5]=32'b01000000000000000000000000000000;
        ph5[26]=32'b00000000000000000000000001000000;
    end

    reg blood_px;
    always @(*) begin
        case (phase)
            3'd0: blood_px = in_box && ph0[dpy][31 - dpx];
            3'd1: blood_px = in_box && ph1[dpy][31 - dpx];
            3'd2: blood_px = in_box && ph2[dpy][31 - dpx];
            3'd3: blood_px = in_box && ph3[dpy][31 - dpx];
            3'd4: blood_px = in_box && ph4[dpy][31 - dpx];
            3'd5: blood_px = in_box && ph5[dpy][31 - dpx];
            default: blood_px = 1'b0;
        endcase
    end

    // bright red fading to dark red
    wire [3:0] blood_r =
        (phase==3'd0) ? 4'hF : (phase==3'd1) ? 4'hF :
        (phase==3'd2) ? 4'hE : (phase==3'd3) ? 4'hC :
        (phase==3'd4) ? 4'hA : 4'h8;
    wire [3:0] blood_g =
        (phase==3'd0) ? 4'h1 : (phase==3'd1) ? 4'h0 :
        (phase==3'd2) ? 4'h0 : 4'h0;

    assign in_blood = blood_px;
    assign r = blood_px ? blood_r : 4'h0;
    assign g = blood_px ? blood_g : 4'h0;
    assign b = 4'h0;
endmodule
```

A.11 — trophy

```
`timescale 1ns / 1ps
module trophy(
    input wire clk,
    input wire active,
    input wire [9:0] hpos,
    input wire [9:0] vpos,
    input wire [9:0] player_x,
    input wire [9:0] player_y,
    output wire in_trophy,
    output wire player_wins,
    output wire [3:0] r,
    output wire [3:0] g,
    output wire [3:0] b
);
    localparam [9:0] TX = 10'd374;
    localparam [9:0] TY = 10'd129;
    localparam [9:0] TW = 10'd16;
    localparam [9:0] TH = 10'd16;

    // 8x8 cup shape
    reg [7:0] trophy_rom [0:7];
    initial begin
        trophy_rom[0] = 8'b01111110;
        trophy_rom[1] = 8'b11111111;
        trophy_rom[2] = 8'b11111111;
        trophy_rom[3] = 8'b01111110;
        trophy_rom[4] = 8'b00111100;
        trophy_rom[5] = 8'b00011000;
        trophy_rom[6] = 8'b00111100;
        trophy_rom[7] = 8'b01111110;
    end

    wire in_box = active &&
        (hpos >= TX) && (hpos < TX + TW) &&
        (vpos >= TY) && (vpos < TY + TH);
    wire [9:0] rx = hpos - TX;
    wire [9:0] ry = vpos - TY;
```

```
38      wire [2:0] col = rx[3:1]; // halve for 2x scaling
39      wire [2:0] row = ry[3:1];
40      wire px_on = trophy_rom[row][7 - col];
41
42      assign in_trophy = in_box && px_on;
43
44      assign player_wins = (player_x + 10'd8 >= TX) &&
45          (player_x <= TX + TW) &&
46          (player_y + 10'd16 >= TY) &&
47          (player_y <= TY + TH);
48
49      // gold colour
50      assign r = in_trophy ? 4'hF : 4'h0;
51      assign g = in_trophy ? 4'hC : 4'h0;
52      assign b = 4'h0;
53  endmodule
```

A.12 — game_over_screen

```
`timescale 1ns / 1ps
module game_over_screen(
    input wire clk,
    input wire active,
    input wire [9:0] hpos,
    input wire [9:0] vpos,
    input wire game_over,
    output reg in_screen,
    output reg [3:0] r,
    output reg [3:0] g,
    output reg [3:0] b
);
    // G A M E SPC O V R
    reg [7:0] font [0:63];
    initial begin
        font[0]=8'h3C; font[1]=8'h66; font[2]=8'hC0; font[3]=8'hC0;
        font[4]=8'hCE; font[5]=8'hC6; font[6]=8'h66; font[7]=8'h3C;
        font[8]=8'h18; font[9]=8'h3C; font[10]=8'h66; font[11]=8'h66;
        font[12]=8'h7E; font[13]=8'h66; font[14]=8'h66; font[15]=8'h66;
        font[16]=8'hC3; font[17]=8'hE7; font[18]=8'hFF; font[19]=8'hDB;
        font[20]=8'hC3; font[21]=8'hC3; font[22]=8'hC3; font[23]=8'hC3;
        font[24]=8'hFE; font[25]=8'hC0; font[26]=8'hC0; font[27]=8'hFC;
        font[28]=8'hC0; font[29]=8'hC0; font[30]=8'hC0; font[31]=8'hFE;
        font[32]=8'h00; font[33]=8'h00; font[34]=8'h00; font[35]=8'h00;
        font[36]=8'h00; font[37]=8'h00; font[38]=8'h00; font[39]=8'h00;
        font[40]=8'h3C; font[41]=8'h66; font[42]=8'hC3; font[43]=8'hC3;
        font[44]=8'hC3; font[45]=8'hC3; font[46]=8'h66; font[47]=8'h3C;
        font[48]=8'hC3; font[49]=8'hC3; font[50]=8'hC3; font[51]=8'hC3;
        font[52]=8'h66; font[53]=8'h66; font[54]=8'h3C; font[55]=8'h18;
        font[56]=8'hFC; font[57]=8'h66; font[58]=8'h66; font[59]=8'h7C;
        font[60]=8'h78; font[61]=8'h6C; font[62]=8'h66; font[63]=8'h66;
    end

    // dark rectangle behind the text
    wire in_box = (hpos >= 10'd190 && hpos < 10'd450 &&
        vpos >= 10'd255 && vpos < 10'd315);

```

```
38      // "GAME OVER" centred on screen
39      localparam [9:0] TX = 10'd248;
40      localparam [9:0] TY = 10'd275;
41
42      wire in_text = (hpos >= TX && hpos < TX + 10'd144 &&
43          vpos >= TY && vpos < TY + 10'd16);
44      wire [9:0] rx = hpos - TX;
45      wire [9:0] ry = vpos - TY;
46      wire [3:0] cidx = rx[7:4];
47      wire [2:0] xbit = rx[3:1]; // 2x scaled
48      wire [2:0] yrow = ry[3:1];
49
50      reg [2:0] cfid;
51      always @(*) case(cidx)
52          4'd0: cfid = 3'd0;
53          4'd1: cfid = 3'd1;
54          4'd2: cfid = 3'd2;
55          4'd3: cfid = 3'd3;
56          4'd4: cfid = 3'd4;
57          4'd5: cfid = 3'd5;
58          4'd6: cfid = 3'd6;
59          4'd7: cfid = 3'd3;
60          4'd8: cfid = 3'd7;
61          default: cfid = 3'd4;
62      endcase
63
64      wire [5:0] faddr = {cfid, yrow};
65      wire fpx = font[faddr][7 - xbit];
66
67      // yellow text on dark navy background
68      always @(posedge clk) begin
69          in_screen <= game_over && active && in_box;
70          if (!active || !game_over) begin
71              r <= 4'h0; g <= 4'h0; b <= 4'h0;
72          end else if (in_text && fpx) begin
73              r <= 4'hF; g <= 4'hF; b <= 4'h0;
74          end else if (in_box) begin
75              r <= 4'h1; g <= 4'h1; b <= 4'h2;
76          end else begin
```

```
77                r <= 4'h0; g <= 4'h0; b <= 4'h0;
78            end
79        end
80    endmodule
```

A.13 — win_screen

```
`timescale 1ns / 1ps
module win_screen(
    input wire clk,
    input wire active,
    input wire [9:0] hpos,
    input wire [9:0] vpos,
    input wire you_win,
    output reg in_screen,
    output reg [3:0] r,
    output reg [3:0] g,
    output reg [3:0] b
);
    // Y O U SPC W I N !
    reg [7:0] font [0:63];
    initial begin
        font[0]=8'hC3; font[1]=8'hC3; font[2]=8'h66; font[3]=8'h3C;
        font[4]=8'h18; font[5]=8'h18; font[6]=8'h18; font[7]=8'h18;
        font[8]=8'h3C; font[9]=8'h66; font[10]=8'hC3; font[11]=8'hC3;
        font[12]=8'hC3; font[13]=8'hC3; font[14]=8'h66; font[15]=8'h3C;
        font[16]=8'hC3; font[17]=8'hC3; font[18]=8'hC3; font[19]=8'hC3;
        font[20]=8'hC3; font[21]=8'hC3; font[22]=8'h66; font[23]=8'h3C;
        font[24]=8'h00; font[25]=8'h00; font[26]=8'h00; font[27]=8'h00;
        font[28]=8'h00; font[29]=8'h00; font[30]=8'h00; font[31]=8'h00;
        font[32]=8'hC3; font[33]=8'hC3; font[34]=8'hC3; font[35]=8'hDB;
        font[36]=8'hDB; font[37]=8'hFF; font[38]=8'h66; font[39]=8'h66;
        font[40]=8'hFF; font[41]=8'h18; font[42]=8'h18; font[43]=8'h18;
        font[44]=8'h18; font[45]=8'h18; font[46]=8'h18; font[47]=8'hFF;
        font[48]=8'hC3; font[49]=8'hE3; font[50]=8'hF3; font[51]=8'hDB;
        font[52]=8'hCF; font[53]=8'hC7; font[54]=8'hC3; font[55]=8'hC3;
        font[56]=8'h18; font[57]=8'h18; font[58]=8'h18; font[59]=8'h18;
        font[60]=8'h18; font[61]=8'h00; font[62]=8'h00; font[63]=8'h18;
    end

    // dark rectangle behind the text
    wire in_box = (hpos >= 10'd180 && hpos < 10'd460 &&
        vpos >= 10'd255 && vpos < 10'd315);

```

```
38      // "YOU WIN!" centred on screen
39      localparam [9:0] TX = 10'd256;
40      localparam [9:0] TY = 10'd275;
41
42      wire in_text = (hpos >= TX && hpos < TX + 10'd128 &&
43          vpos >= TY && vpos < TY + 10'd16);
44      wire [9:0] rx = hpos - TX;
45      wire [9:0] ry = vpos - TY;
46      wire [2:0] cidx = rx[6:4];
47      wire [2:0] xbit = rx[3:1]; // 2x scaled
48      wire [2:0] yrow = ry[3:1];
49
50      reg [2:0] cfid;
51      always @(*) case(cidx)
52          3'd0: cfid = 3'd0;
53          3'd1: cfid = 3'd1;
54          3'd2: cfid = 3'd2;
55          3'd3: cfid = 3'd3;
56          3'd4: cfid = 3'd4;
57          3'd5: cfid = 3'd5;
58          3'd6: cfid = 3'd6;
59          3'd7: cfid = 3'd7;
60          default: cfid = 3'd3;
61      endcase
62
63      wire [5:0] faddr = {cfid, yrow};
64      wire fpx = font[faddr][7 - xbit];
65
66      // yellow text on dark green background
67      always @(posedge clk) begin
68          in_screen <= you_win && active && in_box;
69          if (!active || !you_win) begin
70              r <= 4'h0; g <= 4'h0; b <= 4'h0;
71          end else if (in_text && fpx) begin
72              r <= 4'hF; g <= 4'hF; b <= 4'h0;
73          end else if (in_box) begin
74              r <= 4'h0; g <= 4'h2; b <= 4'h0;
75          end else begin
76              r <= 4'h0; g <= 4'h0; b <= 4'h0;
```

```
77          end
78      end
79  endmodule
```